

HoloWAN®

Network Emulator

Python API 文档

Version 2908, July 2025

1. 文档说明

2. 快速上手

2.1 Python API 安装

2.1.1 Windows 系统安装

2.1.2 MAC 系统安装

2.2 Python 脚本编写

3. 仿真引擎 Engine

3.1 属性

3.2 方法

3.2.1 开启引擎

3.2.2 关闭引擎

3.2.3 重置引擎

3.2.4 保存引擎配置

3.2.5 检查引擎锁

3.2.6 引擎解锁

3.2.7 设置引擎锁新密码

3.2.8 清除引擎锁密码

3.2.9 获取虚链路当前损伤配置

3.2.10 重置虚拟链路

3.2.11 添加虚拟链路

3.2.12 删除虚拟链路

3.2.13 启用虚拟链路

3.2.14 禁用虚拟链路

3.2.15 应用虚拟链路配置

3.2.16 应用报文分类器配置

3.2.17 应用损伤

3.2.18 向报文分类器添加规则

3.2.19 更新引擎配置

3.2.20 设置线路选择

3.3 Sequential容器

3.4 返回值与错误码

4. 报文分类 PacketClassifier

4.1 属性

4.2 方法

4.2.1 添加分类规则

- 4.2.2 修改分类规则
- 4.2.3 删除分类规则
- 4.2.4 在指定位置添加分类规则
- 4.2.5 清空引擎所有分类规则
- 4.2.6 获取引擎所有分类规则
- 4.2.7 其他方法

4.3 分类规则

- 4.3.1 IPv4 地址分类规则
- 4.3.2 IPv6 地址分类规则
- 4.3.3 MAC 地址分类规则
- 4.3.4 MPLS 分类规则
- 4.3.5 PPPoE 分类规则
- 4.3.6 RawByte 分类规则
- 4.3.7 TCP 端口号分类规则
- 4.3.8 SCTP 端口号分类规则
- 4.3.9 UDP 端口号分类规则
- 4.3.10 VLAN 分类规则
- 4.3.11 组合分类规则
- 4.3.12 设置分类规则的名称

5. 损伤参数 Path

5.1 属性

5.2 方法

- 5.2.1 重置链路
- 5.2.2 更新链路

5.3 损伤

- 5.3.1 带宽 (Bandwidth)
 - 5.3.1.1 固定带宽 (Fixed)
 - 5.3.1.2 曲线控制带宽变化 (Jitter)
 - 5.3.1.3 令牌桶 (TokenBucket)
 - 5.3.1.4 双向带宽限制模式 (Bidirectional)
- 5.3.2 背景流量 (BackgroundUtilization)
 - 5.3.2.1 禁用背景流量
 - 5.3.2.2 随机模式 (Random)
 - 5.3.2.3 PCAP 模式 (PCAP)
- 5.3.3 队列深度 (QueueLimit)
 - 5.3.3.1 简单模式 (Simple)
 - 5.3.3.2 尾部丢弃 (DropTail)
 - 5.3.3.3 随机早检测 (RED)
- 5.3.4 帧开销 (FrameOverhead)
 - 5.3.4.1 以太网 24 Ethernet模式
 - 5.3.4.2 以太网4 Ethernet模式
 - 5.3.4.3 自定义模式 (Custom)

5.3.5 报文修改 (Modify)

5.3.5.1 禁用报文修改

5.3.5.2 全部修改 (Normal)

5.3.5.3 周期性修改 (Cycle)

5.3.5.4 固定概率修改 (Random)

5.3.5.5 多范围模式 (Range)

5.3.5.6 插入模式 (Insert)

5.3.5.7 交换模式 (Exchange)

5.3.5.8 次数修改模式 (Count)

5.3.6 大包分片 (MTU)

5.3.6.1 禁用大包分片

5.3.6.2 普通模式 (MTULimit)

5.3.7 时延 (Delay)

5.3.7.1 常量 (Constant)

5.3.7.2 均匀分布 (Uniform)

5.3.7.3 正态分布 (Normal)

5.3.7.4 自定义分布 (Custom)

5.3.7.5 抖动 (Jitter)

5.3.7.6 伽马分布 (Gamma)

5.3.7.7 累计突发时延 (AccumulateBurst)

5.3.7.8 自定义文件模式 (Customized File)

5.3.8 丢包 (Loss)

5.3.8.1 固定概率丢包 (Random)

5.3.8.2 周期性丢包 (Cycle)

5.3.8.3 突发丢包 (Burst)

5.3.8.4 双通道模式丢包 (GilbertElliott)

5.3.8.5 曲线控制丢包率 (Jitter)

5.3.8.6 四状态马尔可夫丢包 (Markov)

5.3.8.7 次数丢包 (Count)

5.3.9 误码率 (Corruption)

5.3.9.1 全报文误码 (BER)

5.3.9.2 多范围误码 (Range)

5.3.9.3 错包率 (Packet)

5.3.9.4 次数误码 (Count)

5.3.10 乱序 (Reordering)

5.3.10.1 普通模式 (Normal)

5.3.10.2 曲线控制乱序概率 (Jitter)

5.3.10.3 周期模式 (Cycle)

5.3.11 重复帧 (Duplication)

5.3.11.1 普通模式 (Normal)

5.3.11.2 曲线控制重复帧出现概率 (Jitter)

5.3.12 重定向 (Redirect)

5.3.13 损伤组 (Impairments)

5.3.14 线路选择

5.3.14.1 RouteSelect

5.3.14.2 RouteSelectClient

- 5.3.14.3 RouteSelectServer
- 5.3.14.4 RouteSelectISP
- 5.3.14.5 RouteSelectNetType

6. 设置 Admin

6.1 HoloWAN 设置

- 6.1.1 重启设备
- 6.1.2 获取设备首选项配置
- 6.1.3 设置设备首选项
- 6.1.4 获取Ethernet状态信息
- 6.1.5 获取设备的网络配置
- 6.1.6 修改设备的网络配置

6.2 首选项（Preference）

6.3 Ethernet状态（EthernetStatus）

- 6.3.1 网口状态（PortStatus）

6.4 网络配置（NetworkConfig）

6.5 时间同步（Time Sync）

- 6.5.1 获取HoloWAN时间
- 6.5.2 时间同步

7. 抓包分析 Pixel

7.1 Pixel 过滤器设置

7.2 Pixel 设置

- 7.2.1 设置过滤器配置
- 7.2.2 开始抓包
- 7.2.3 停止抓包
- 7.2.4 清空抓包器
- 7.2.5 导出 pcap 文件
- 7.2.6 设置抓包对象
- 7.2.7 获取所有pixel文件信息
- 7.2.8 上传pixel文件
- 7.2.9 释放pixel文件
- 7.2.10 删除pixel文件

7.3 Pixel 过滤规则

- 7.3.1 IPv4 地址过滤规则
- 7.3.2 IPv6 地址过滤规则
- 7.3.3 报文长度规则
- 7.3.4 MAC 地址过滤规则
- 7.3.5 TCP 端口号过滤规则
- 7.3.6 UDP 端口号过滤规则
- 7.3.7 SCTP 端口号过滤规则
- 7.3.8 VLAN 过滤规则

7.3.9 Type 损伤类型过滤规则

7.3.10 Direction 方向过滤规则

7.3.11 Path 过滤规则

7.3.12 重置过滤规则

7.4 设置抓包对象 (PixelMode)

7.5 获取 pixel 抓包总数 (get_pixel_capture_pkt_count)

7.6 获取 pixel 数据包信息 (get_pixel_capture_pkt_data)

8. 损伤回放 PlackBack

8.1 上传回放文件

8.2 获取已上传的回放文件列表

8.3 删除回放文件

8.4 获取回放文件内容

8.5 应用回放文件到虚拟链路

8.6 释放回放文件

8.7 获取回放状态

8.8 设置回放状态

8.8.1 从当前时刻开始或暂停回放

8.8.2 指定时刻开始或暂停回放

8.9 设置回放开关

9. 统计数据

9.1 统计数据 Statistics

9.1.1 清理历史统计数据

9.1.2 将历史数据保存为 csv 文件

9.1.3 获取所有 pah 的历史数据

9.2 获取指定虚拟链路的统计数据

附录一：传输内容解析

附录二：错误码信息

附录三：统计数据类型列举

附录四：损伤参数的配置值范围

附录五：参数取值列举

附录六：HoloWAN Python API 脚本示例

1. 文档说明

本文档是关于 HoloWAN Python API 的介绍。通过这些 API，您可以编写 Python 脚本对 HoloWAN 实现修改 Path（虚拟链路）、配置 Path（虚拟链路）的损伤参数、配置 Classifier（报文分类器）等操作。

自 2900 版本起，HoloWAN 引入了全新的 Python API。此 Python API 完全保持旧版 API 的兼容性。因此，您无需更改任何代码，可以继续使用 HoloWAN 的旧版 Python API。相比 2900 之前的版本，新版本的 API 对功能进行了扩充，进一步提升了它的灵活性。

请参阅 [附录六](#)，其中包含用 HoloWAN Python API 编写 Python 脚本的示例。

2. 快速上手

HoloWAN Python API 支持 Python 3.8 及以上版本。若您的电脑中存在不同版本的 Python，在安装 HoloWAN Python API 时，请注意将其安装至正确版本的 Python 下。

2.1 Python API 安装

HoloWAN Python API 的安装包中为用户提供了快捷安装程序，安装程序在不同的操作系统上的使用方式也不一样。

2.1.1 Windows 系统安装

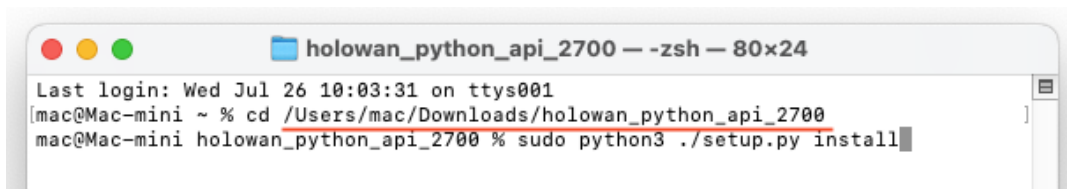
1. 打开 HoloWAN Python API 文件夹，双击运行 install.bat 程序。

名称	修改日期	类型	大小
API文档	2023/6/30 14:44	文件夹	
holowan	2023/6/7 16:10	文件夹	
install	2023/2/28 15:05	Windows 批处理...	1 KB
PKG-INFO	2023/2/28 15:05	文件	1 KB
README	2023/2/28 15:05	文本文档	2 KB
setup	2023/2/28 15:05	CFG 文件	1 KB
setup.py	2023/6/1 14:49	Python.File	1 KB

2. 双击程序后，将弹出 Windows 命令窗口并自动运行安装程序。
3. 观察命令窗口信息，如无报错，则 Holowan Python API 安装成功。
4. 如安装失败，请联系售后人员解决。

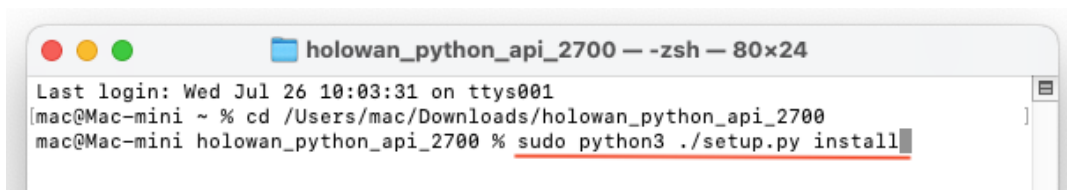
2.1.2 Mac 系统安装

1. 打开终端，使用 `cd /HoloWAN Python API 文件夹路径` 命令进入 Python API 文件目录。



```
holowan_python_api_2700 — -zsh — 80x24
Last login: Wed Jul 26 10:03:31 on ttys001
[mac@Mac-mini ~ % cd /Users/mac/Downloads/holowan_python_api_2700
mac@Mac-mini holowan_python_api_2700 % sudo python3 ./setup.py install
```

2. 执行 `sudo python3 ./setup.py install` 命令，运行 `setup.py` 脚本。

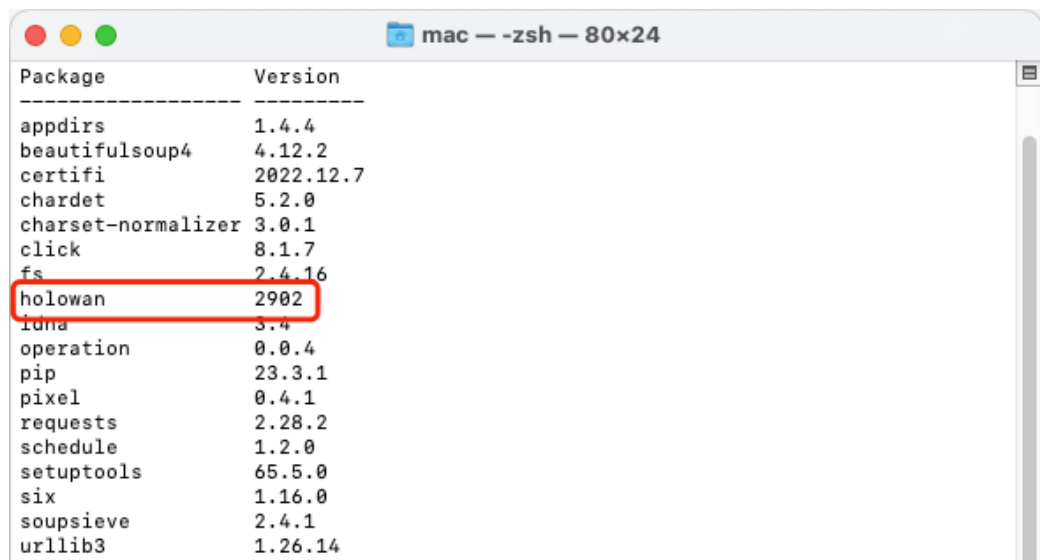


```
holowan_python_api_2700 — -zsh — 80x24
Last login: Wed Jul 26 10:03:31 on ttys001
[mac@Mac-mini ~ % cd /Users/mac/Downloads/holowan_python_api_2700
mac@Mac-mini holowan_python_api_2700 % sudo python3 ./setup.py install
```

- 3. 观察终端窗口信息，如无报错，则 Holowan Python API 安装成功。
- 4. 如安装失败，请联系售后人员解决。

2.1.3 检查是否成功安装

安装完成后，打开终端，执行 `pip3 list` 命令，列出所有已安装的 Python 库。检查是否已经成功安装对应版本的 HoloWAN Python SDK。

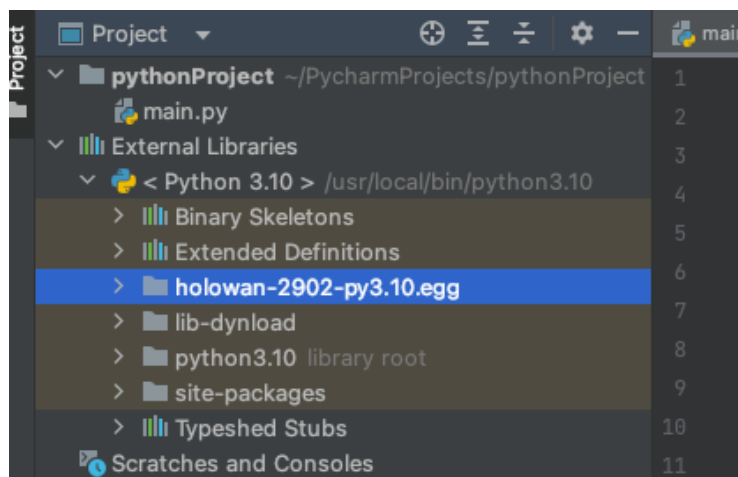


Package	Version
appdirs	1.4.4
beautifulsoup4	4.12.2
certifi	2022.12.7
chardet	5.2.0
charset-normalizer	3.0.1
click	8.1.7
fs	2.4.16
holowan	2902
idna	3.4
operation	0.0.4
pip	23.3.1
pixel	0.4.1
requests	2.28.2
schedule	1.2.0
setuptools	65.5.0
six	1.16.0
soupsieve	2.4.1
urllib3	1.26.14

2.2 Python 脚本编写

Python API 安装完成后，即可开始编写 Python 脚本。

步骤一、使用 IDE 编写和运行 HoloWAN 相关脚本之前，请确认 IDE 中存在正确的 HoloWAN 库。可以在 IDE 导航 **External Libraries**（外部库）中检查是否存在正确版本的 holowan 库。



Pycharm IDE 导航栏

步骤二、在 Python 代码中导入 HolowAN 库：

```
from holowan.v2.engine import Engine # 导入引擎模块
from holowan.v2.engine.path import DelayGamma # 导入损伤模块
from holowan.v2.engine.classifier import IPv4Rule # 导入报文过滤器模块
```

步骤三、获取待操作的引擎，如下代码获取了 192.168.1.199 设备的引擎1：

```
holowan_ip = "192.168.1.199" # HolowAN 管理端口 IP地址
holowan_port = "8080" # HolowAN 端口号，默认为 8080
engine_id = 1 # 下发损伤配置的引擎ID, engine1 ID 为 1, engine2 ID 为 2, 以此类推
engine = Engine(holowan_ip, holowan_port, engine_id) # 创建Engine对象
```

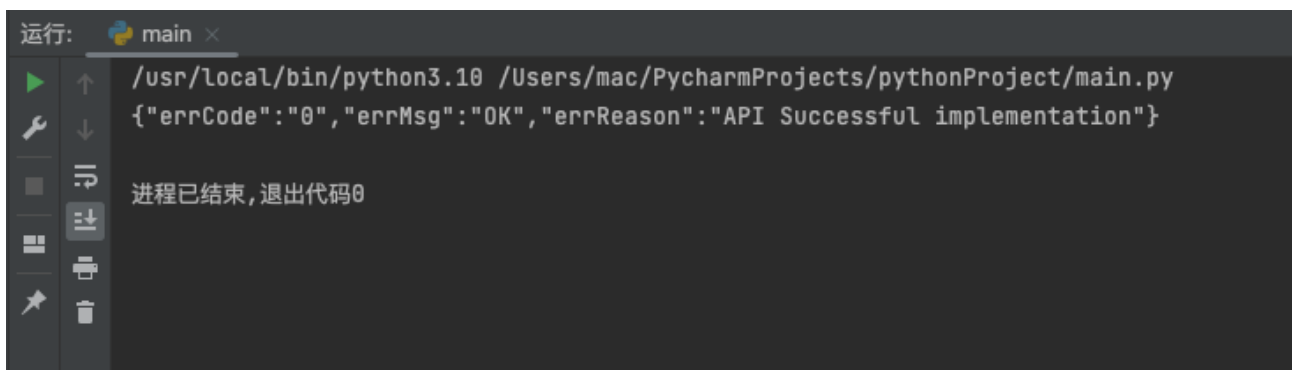
步骤四、设置报文过滤器和损伤，如下代码设置了IPv4地址分类规则和Gamma分布时延：

```
# 设置 IPv4地址分类规则
ipv4 = IPv4Rule(src="192.168.1.10", smask=32, dst="any", dmask=32, tos="any", action=1)
# 将上述 IPv4地址分类规则应用到 HolowAN 的 port1 业务口
result = engine.apply_rule_to_classifier(ipv4, port=1)
print(result) # result 可以直接判断分类规则是否应用成功

# 设置 Gamma 分布时延损伤参数
impairment = DelayGamma(shape=21.1, scale=1.1, enable_reordering=1)
path_id=1 # 下发损伤的 path id, path1 id 为 1, path2 id 为 2, 以此类推
direction= 1 # 设置损伤的 path 的方向, 1 表示只在 port1 to port2 方向设置损伤
# 应用损伤设置
result = engine.apply_impairment(impairment, path_id, direction)

print(result) # result 可以直接判断损伤是否应用成功
```

步骤五、运行脚本并查看 [返回值](#)。脚本运行成功，会返回 0，并提示“API Successful implementation”。脚本运行失败，会返回 负数，并在 errReson 中提示失败原因。



The image shows the PyCharm Run window. At the top, it says '运行: main x'. Below that, the command executed is `/usr/local/bin/python3.10 /Users/mac/PycharmProjects/pythonProject/main.py`. The output is a JSON object: `{"errCode": "0", "errMsg": "OK", "errReason": "API Successful implementation"}`. At the bottom, it says '进程已结束, 退出代码0'.

```
运行: main x
/usr/local/bin/python3.10 /Users/mac/PycharmProjects/pythonProject/main.py
{"errCode": "0", "errMsg": "OK", "errReason": "API Successful implementation"}
进程已结束, 退出代码0
```

Pycharm 运行结果

脚本运行成功后可以回到 web GUI 界面检查执行结果，刷新网页，即可观察到 HoloWAN 配置发生改变。

3. 引擎 Engine

Engine 类是 HoloWAN Python API 的核心类，所有与仿真引擎相关的操作，例如设置报文分类器、配置Path损伤等，都需通过 Engine 类的实例对象来执行。

一对物理网口组成一个仿真引擎，不同型号的 HoloWAN 拥有的仿真引擎数目也不一样。

举例：

```
from holowan.v2.engine import Engine
holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080" # HoloWAN 端口号，默认为 8080
engine_id = 1 # 下发损伤配置的引擎ID, engine1 ID 为 1, engine2 ID 为 2, 以此类推
engine1 = Engine(holowan_ip, holowan_port, engine_id) # 创建Engine对象
```

参数：

参数	类型	参数含义
holowan_ip	str	HoloWAN 管理端口 IP地址。
holowan_port	str	HoloWAN 端口号，默认为 8080。
engine_id	int	下发损伤配置的引擎ID，engine1 ID 为 1，engine2 ID 为 2，以此类推。

注意： Engine 类具备将配置应用到 HoloWAN 的能力，也具备从 HoloWAN 获取最新配置的能力。

3.1 属性

- **holowan_ip**
HoloWAN 管理端口的 IP 地址。只读属性。
- **holowan_port**
HoloWAN 端口。默认为 8080。只读属性。
- **engine_id**
仿真引擎的 id。只读属性。
- **packet_classifier**
该属性为 [PacketClassifier](#) 的实例对象。可读可写。
- **paths_info**
dict 类型，为当前引擎下存在的虚拟链路的信息。key 为 path_id，int 类型；value 为 path 的信息，dict 类型，aths_info 包含两个参数：
 - i. path_name: str类型，虚拟链路的名称。
 - ii. is_enable: int类型，是否启用虚拟链路。1表示开启，2表示关闭。

注意：
当通过属性访问报文分类器或 paths_info 时，将不会从 HoloWAN 获取最新的配置信息。因此，所获取的报文

分类器或paths_info 可能与 HoloWAN 内的报文分类器或虚拟链路信息存在差异。如需获取最新的报文分类器或paths_info，请首先调用 Engine.update() 方法。

3.2 方法

3.2.1 开启仿真引擎 emulation_on()

仿真引擎处于 **开启** 状态时，所有损伤配置才能下发成功并按照设置对报文构造损伤。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"        # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 开启仿真引擎
result = engine.emulation_on()

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

3.2.2 关闭仿真引擎 emulation_off()

仿真引擎处于 **关闭** 状态时，所有进入 HoloWAN 的流量都不会受到任何损伤，并直接转发出 HoloWAN。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"        # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 关闭仿真引擎
result = engine.emulation_off()

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

3.2.3 重置引擎 reset_engine()

仿真引擎重置后，所有的报文分类规则配置，损伤配置都将被删除，并恢复到出厂状态。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.reset_engine() # 重置引擎

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

3.2.4 保存引擎 save_engine()

保存仿真引擎可以将仿真引擎上已经设置的报文分类规则，损伤配置，链路数目等配置保存为开机默认状态。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.save_engine() # 保存引擎

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

3.2.5 检查引擎锁 check_lock()

检查仿真引擎是否上锁。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.check_lock() # 检查引擎是否上锁

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

3.2.6 引擎开锁 unlock()

若仿真引擎已上锁，则使用新终端访问已上锁的仿真引擎之前需要先开锁。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

password = 1234
result = engine.unlock(password) # 开锁

print(result)
```

参数：

参数	类型	参数含义
password	int	开锁密码

返回值：

- [HoloWANReturn](#) 对象。

3.2.7 设置引擎锁新密码 set_new_password()

修改已上锁的仿真引擎的引擎锁密码。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

password = 1234
new_password = 4321
result = engine.set_new_password(password, new_password) # 重置引擎锁密码

print(result)

```

参数	类型	参数含义
password	int	引擎锁源密码
new_password	int	引擎锁新密码

返回值：

- [HoloWANReturn](#) 对象。

3.2.8 清除引擎锁密码 clear_password()

清除已上锁的仿真引擎的引擎锁密码，引擎锁将恢复到未上锁状态。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.clear_password() # 清除引擎锁密码

print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

3.2.9 获取虚链路当前损伤配置 get_path_by_id()

通过 path 的 ID 来获取指定的 path 对象，该 path 上的损伤配置也将一同被获取。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

path_id = 1
path1 = engine.get_path_by_id(path_id) # 通过 pathid 来获取指定的 path 对象

# 打印 path1 上的所有损伤配置
print(path1)

```

返回值：

- 指定的 Path 对象

3.2.10 重置虚拟链路 reset_path(path_id:int)

重置 path，将 path 重置为默认无损状态。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

path_id = 1
result = engine.reset_path(path_id) # 重置指定的 Path

print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

3.2.11 添加虚拟链路 add_path()

添加一条 path，添加的 path 默认处于启用状态。

举例：


```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.add_path() # 添加一条 Path

print(result)
```

返回值：

- 一个整数。如果成功添加 path，返回的是新添加的 path 的 id；如果添加失败，返回 -1。

3.2.12 删除虚拟链路 remove_path()

删除指定的 path。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

path_id = 1
path_name = "PATH 1"
result = engine.remove_path(path_id,path_name) # 删除指定的 Path

print(result)
```

参数：

参数	类型	参数含义
path_id	int	待删除的 path 的 id
path_name	str	待删除的 path 的名称

返回值：

- [HoloWANReturn](#) 对象。

3.2.13 启用虚拟链路 enable_path ()

启用 path_id 和 path_name 指定的虚拟链路。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

path_id = 1
path_name = "PATH 1"
result = engine.enable_path(path_id,path_name) # 启用指定的 Path

print(result)
```

参数：

参数	类型	参数含义
path_id	int	待启用的 path 的 id
path_name	str	待启用的 path 的名称

返回值：

- [HoloWANReturn](#) 对象。

3.2.14 禁用虚拟链路 disable_path()

禁用 path_id 和 path_name 指定的虚拟链路。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

path_id = 1
path_name = "PATH 1"
result = engine.disable_path(path_id,path_name) # 禁用指定的 Path

print(result)
```

参数：

参数	类型	参数含义
path_id	int	待禁用的 path 的 id
path_name	str	待禁用的 path 的名称

返回值：

- [HoloWANReturn](#) 对象。

3.2.15 应用虚拟链路配置 `apply_path_configuration()`

将指定 Path 的配置应用到 HoloWAN。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"          # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)
path_id = 1
path1 = engine.get_path_by_id(path_id)
# 对 path1 进行配置
# ...
result = engine.apply_path_configuration(path1) # 将指定的 Path 配置应用到 HoloWAN

print(result)
```

参数：

参数	类型	参数含义
path	Path类实例对象	待配置的 path

返回值：

- [HoloWANReturn](#) 对象。

3.2.16 应用报文分类器配置 `apply_classifier_changes()`

将报文分类器的配置应用到 HoloWAN。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)
# 对 engine.packet_classifier 进行配置
# ...
result = engine.apply_classifier_changes() # 应用报文分类器配置

print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

3.2.17 应用损伤 `apply_impairment()`

将设置的损伤参数应用到指定的 Path 上。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

impairment=DelayGamma(shape=21.1,scale=1.1,enable_reordering=1) # 设置损伤类型以及损伤参数
path_id=1      # 下发损伤的 path id, path1 id 为 1, path2 id 为 2, 以此类推
direction= 1   # 设置损伤的 path 的方向, 1 表示只在 port1 to port2 方向设置损伤

result = engine.apply_impairment(impairment,path_id,direction) # 应用设置的损伤

if result:      # result 可以直接进行true/false判断, 判断损伤是否应用成功
    print(result.err_code,result.err_msg,result.err_reason)

```

参数：

参数	类型	参数含义
impairment	损伤实例对象	待应用的损伤
path_id	int	需要设置损伤的 Path 的 id
direction	int	损伤配置的方向。1 表示配置 port1 -> port2 方向上的损伤；2 表示配置 port2 -> port1 方向上的损伤；3 表示配置上述两个方向上的损伤

返回值：

- [HoloWANReturn](#) 对象。

注意： 当您使用Engine.apply_impairment方法设置损伤时，无需调用Engine.apply_path_configuration方法。

3.2.18 向报文分类器添加规则 apply_rule_to_classifier()

将报文分类规则添加到指定的端口上，新的分类规则将被添加到原有分类规则的最后面，新的分类规则优先级最低。

举例：

```
from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

rule = raw1(layer=2, offset=0, mask="0x0", value="0x0") # 设置分类规则
port = 1
result = engine.apply_rule_to_classifier(rule,port) # 应用设置的报文分类器

if result:      # result 可以直接进行true/false判断，判断报文分类规则是否应用成功
    print(result.err_code,result.err_msg,result.err_reason)
```

参数：

参数	类型	含义
rule	Rule	分类规则子类实例对象，待设置的分类规则
port	int	待添加分类规则的网口 id。1 表示port1；2 表示 port2

返回值：

- [HoloWANReturn](#) 对象。

注意： 当您使用Engine.apply_rule_to_classifier方法设置规则时，无需调用Engine.apply_classifier_changes方法。

3.2.19 更新引擎配置 Engine.update()

该方法能够从 HoloWAN 获取最新的配置信息，来更新 Engine 类的实例对象。

举例：

```

from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 在创建 Engine 的实例对象时，会从 HolowAN 获取指定引擎的配置信息。
engine1 = Engine(holowan_ip, holowan_port, engine_id)
# ...
# 如果此期间通过 Web GUI 更改了该引擎的配置，则会出现 engine1 和实际的仿真引擎配置不一致的情况
# 可以使用 update() 方法，从 HolowAN 获取最新的引擎配置信息，以更新 engine1
engine1.update()

```

3.2.20 线路选择 Engine.set_route_select()

在指定的 Path 上设置线路选择参数。

举例：

```

from holowan.v2.engine import Engine
from holowan.v2.engine.path._route_select import (RouteSelect, RouteSelectClient,
                                                    RouteSelectServer, RouteSelectNetType,
                                                    RouteSelectISP)

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 给 path1 设置线路选择参数
rs = RouteSelect(
    holowan_ip = holowan_ip, holowan_port = holowan_port,
    client = RouteSelectClient.BeiJing, server = RouteSelectServer.ChengDu,
    network_type = RouteSelectNetType.WIFI, isp = RouteSelectISP.CHINA_TELECOM,
    uplink_direction = 1)
# 应用配置
result = engine1.set_route_select(path_id = 1, route_select = rs)

print(result)

```

参数：

参数	类型	含义
path_id	int	待设置路线选择的 path 的id
route_select	RouteSelect类实例对象	待设置的路线选择。详见 RouteSelect 对象。

返回值：

- [HoloWANReturn](#) 对象。

3.3 Sequential 容器

Sequential 是报文分类器规则 Rule 以及虚拟链路 Path 的容器。容器内的元素为键值对 [key,value]，其中 key 为 str 类型，value 为 Rule 或 Path 类型。默认的 key 是元素传入次序的 str 形式，如第一个元素，如果没有指定 key，则默认为 '0'(即元素索引的字符串形式)。

可以使用索引获取容器内的元素，假设有容器rules，可以通过 rules[0]的方式获取第一个元素。

通过 Sequential 容器，您可以添加多条规则，规则将按照参数的传入顺序存储在容器内。将配置应用到 HoloWAN 时，规则也将按照传入顺序应用到 HoloWAN 相应的仿真引擎上。

注意： 容器的索引从 0 开始。容器内元素的 key 是唯一的。

方法：

- **add_item(key:str, item)**
item将被添加到容器的末尾，并且 key 为传入的 key 参数。
- **get_item_by_key(key:str)**
可以通过 key 来获取容器内的元素。
- **remove_item_by_key(key:str)**
可以通过 key 删除容器内的元素。
- **insert(index:int, item)**
在index的位置插入元素。元素的key默认为元素索引的字符串形式。
- **append(item)**
在容器尾部添加元素，key 的值将是元素所在位置的 str 类型。
- **rearrange(order:list)**
传入一个 order 列表，容器内的元素将按照列表内的索引顺序重新排列。重排后的容器内元素数量等于 order 列表的元素个数。例如容器内有4个元素a, b, c, d，索引为0, 1, 2, 3，传入的 order 列表为[0,3,1]，重排后的容器内元素为：a, d, b。

举例：

```

from holowan.v2.engine import Engine
from holowan.v2.engine import Sequential
from holowan.v2.engine.classifier import TCPRule,MACRule,PPPoERule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 按顺序将分类规则添加到 Sequential 容器中
engine1.packet_classifier.port1 = Sequential(
    TCPRule(src=123, dst=456, check_version=0, action=1),
    MACRule(src="any", dst="any", type="any", action=1),
    PPPoERule(sid="any", code="any", action=1)
)
# 此时 engine1.packet_classifier.port1 的规则为上述三条，顺序为参数的传入顺序
# 将分类规则应用到 HoloWAN 相应的仿真引擎
result = engine1.apply_classifier_changes()

# 打印, result 判断是否应用成功
print(result)

```

3.4 返回值与错误码 HoloWANReturn

当调用 API 接口对 HoloWAN 进行操作、配置时，后端会返回错误码，错误信息以及错误原因的信息。这三者在 Python API 中表现为 API（部分接口）调用的返回值 HoloWANReturn。详细错误码信息请参考 [附录二](#)。

当 API 调用成功时，错误码为非负的值。

您可以直接打印 result，判断 API 调用的结果是否成功。亦可使用 if 语句对 HoloWANReturn 进行判断。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

result = engine.emulation_on()

# 打印 result 判断是否应用成功
print(result)

```

属性：

- **err_code：** 错误码，由 6 位数字组成，具体含义见 [附录二](#)。
- **err_msg：** 错误信息。

- **err_reason:** 错误原因，造成错误的具体原因。

4 报文分类器 PacketClassifier

报文分类器类。HoloWAN 每个 Engine 上都有独立的报文分类器，报文分类器可以根据报文的 IP 地址、MAC 地址、TCP/UDP/SCTP 端口号等报文特征对报文进行分类。

符合分类规则的报文将会进行进行3类操作。

- 1、分配到指定的虚拟链路上；
- 2、Bypass，报文不受任何损伤，直接转发出 HoloWAN；
- 3、Drop，报文直接丢弃。

注意： PacketClassifier 类自身不具备将配置应用到 HoloWAN 的能力，也不具备从 HoloWAN 获取最新配置的能力。

举例：

```
from holowan.v2.engine.classifier import PacketClassifier
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建报文分类器
classifier = PacketClassifier()

# 当通过 Engine 获取报文分类器时，该 Engine 内原有的报文分类器的配置信息可以被 PacketClassifier 获取
engine1 = Engine(holowan_ip,holowan_port,engine_id)
classifier = engine1.packet_classifier

# 打印报文分类器中已设置的分类规则
print(classifier)
```

4.1 属性

HoloWAN 每个 Engine 上包含两个 port。以 Engine1 为例，Engine1 包含 port1 和 port2，Engine2 包含 port3 和 port4，以此类推。其中 port1 上的分类规则匹配的是从 port1 网口进入 HoloWAN 的报文，port2 上的分类规则匹配的是从 port2 网口进入 HoloWAN 的报文。

- **port1：** Sequential 容器实例对象，表示匹配从 port1 网口进入 HoloWAN 的报文。容器内的元素为分类规则 Rule 的实例对象。该属性可读可写。代表HoloWAN 的网口1，3，5，...
- **port2：** Sequential 容器实例对象，表示匹配从 port2 网口进入 HoloWAN 的报文。容器内的元素为分类规则 Rule 的实例对象。该属性可读可写。代表HoloWAN 的网口2，4，6，...

注意： 每个 port 对应设备上的一个物理网口，除某些型号外，大部分型号的 HoloWAN 的物理网口从左到右依次为 port1、port2、port3、port4、port5、port6。

4.2 方法

4.2.1 添加分类规则

apply_rule_to_classifier(rule:Rule, port:int) 方法可以向 HoloWAN 指定的 port 添加一条规则，规则将添加在该 port 的末尾。

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv4Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 设置 IPv4 地址分类规则参数
ipv4 = IPv4Rule(src="any", smask=32, dst="any", dmask=32, tos="any", action=1)
# 将 IPv4 地址分类规则应用到 port1 上
result = engine.apply_rule_to_classifier(ipv4,port=1)

# 将报文分类器的修改应用到HoloWAN上
result = engine.apply_classifier_changes()

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
rule	Rule	分类规则的名称
port	int	Sequential 容器实例对象，如 port1 表示匹配从 port1 网口进入 HoloWAN 的报文

返回值：

- [HoloWANReturn](#) 对象。

4.2.2 修改分类规则

modify_rule_by_idx(port:int, index:int, rule:Rule) 方法可以修改指定位置的报文分类规则，即报文分类器指定 port 的第 index 条分类规则将被传入的 rule 替换。

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv4Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 设置一条新的 rule
ipv4 = IPv4Rule(src="any", smask=32, dst="any", dmask=32, tos="any", action=1)
# 将 port1 的第一条分类规则替换为上述 IPv4地址分类规则
engine.packet_classifier.modify_rule_by_idx(port=1, index=0, rule=ipv4)

# 将报文分类器的修改应用到HoloWAN上
result = engine.apply_classifier_changes()

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
port	int	Sequential 容器实例对象，如 port1 表示匹配从 port1 网口进入 HoloWAN 的报文
index	int	分类规则的索引号，index = 分类规则的序号 - 1，如第一条分类规则的 index 为 0
rule	Rule	分类规则的名称

返回值：

- [HoloWANReturn](#) 对象。

4.2.3 删除分类规则

remove_rule(port:int,index:int) 方法可以删除指定的报文分类规则。

```

from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv4Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 删除 port1 的第1条规则，第一条规则的index记为0
engine.packet_classifier.remove_rule(port=1, index=0)
# 将报文分类器的修改应用到HoloWAN上
result = engine.apply_classifier_changes()

# 打印 result 判断是否应用成功
print(result)

```

参数：

参数	类型	参数含义
port	int	Sequential 容器实例对象，如 port1 表示匹配从 port1 网口进入 HoloWAN 的报文
index	int	分类规则的索引号，index = 分类规则的序号 - 1，如第一条分类规则的 index 为 0

返回值：

- [HoloWANReturn](#) 对象。

4.2.4 在指定位置添加分类规则

insert_rule(port:int,index:int,rule:Rule) 方法可以在指定的位置添加一条分类规则。该方法将新规则插入到索引号为 index 的规则前面，同时位于新规则之后的所有规则都将自动下移动一位。

```

from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import TCPRule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 需要增加的分类规则
tcprule = TCPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
                  check_version=0, action=1)
# 将上述 TCP 端口号分类规则插入到 port1 的 第一条规则前
engine.packet_classifier.insert_rule(port=1,index=0,rule=tcprule)

# 将报文分类器的修改应用到 HoloWAN 上
result = engine.apply_classifier_changes()

print(result)

```

参数：

参数	类型	参数含义
port	int	Sequential 容器实例对象，如 port1 表示匹配从 port1 网口进入 HoloWAN 的报文
index	int	分类规则的索引号，index = 分类规则的序号 - 1，如第一条分类规则的 index 为 0
rule	Rule	分类规则的名称

返回值：

- [HoloWANReturn](#) 对象。

4.2.5 清空引擎所有分类规则

reset() 方法可以将报文分类器两个 port 的所有规则清空，该方法不具备将配置应用到 HoloWAN 的能力，也不具备从 HoloWAN 获取最新配置的能力，只更新其 XML。

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv4Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"

# 确定需要操作的 engine
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

# 删除 engine1 中所有的报文分类规则
engine.packet_classifier.reset()
# 将报文分类器的修改应用到HoloWAN上
result = engine.apply_classifier_changes()

# 打印 result 判断是否应用成功
print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

4.2.6 获取引擎所有分类规则

engine 初始化时会获取该 engine 下所有的分类规则。

```
from holowan.v2.engine.classifier import PacketClassifier
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建报文分类器
classifier = PacketClassifier()

# 当通过 Engine 获取报文分类器时，该 Engine 内原有的报文分类器的配置信息可以被 PacketClassifier 获取
engine1 = Engine(holowan_ip,holowan_port,engine_id)
classifier = engine1.packet_classifier
# 打印报文分类器中已设置的分类规则
print(classifier)

# 获取 engine1 port1 的所有分类规则
Port1 = engine1.packet_classifier.port1
# 打印 Port1 中已设置的分类规则
print(Port1)
```

4.2.7 其他方法

1、attachTo(engine_id:int)

每个仿真引擎都是独立的，拥有各自独立的报文分类器。该方法只是将 PacketClassifier 实例对象标记为engine_id，修改报文分类器 XML 节点，不会将该 PacketClassifier 实例对象的配置应用到 HoloWAN。

参数：

参数	类型	参数含义
engine_id	int	HoloWAN 的引擎 id，Engine1 id 为1， Engine2 id 为 2 ，以此类推

返回值：

- [HoloWANReturn](#) 对象。

2、update()

该方法不具备将配置应用到 HoloWAN 的能力，也不具备从 HoloWAN 获取最新配置的能力。该方法只更新其 XML。

3、rearrange_rules(port:int,order:list)

该方法可以调整报文分类规则的优先级。传入一个 order 列表，HoloWAN 指定 port 的元素将按照 order 列表内的索引顺序重新排列。重排后的规则数量等于 order 列表的元素个数。

例如容器内有 4 条分类规则 a, b, c, d，索引为 0, 1, 2, 3，传入的 order 列表为[0,3,1,2]，重排后的容器内规则为：a, d, b, c。

举例：

```
# 将分类规则重新排序，调整优先级
engine.packet_classifier.rearrange_rules(port=1, order=[0, 3, 1, 2])
# 将报文分类器的修改应用到HoloWAN上
result = engine.apply_classifier_changes()
# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
port	int	Sequential 容器实例对象，如 port1 表示匹配从 port1 网口进入 HoloWAN 的报文
order	list[int]	报文分类规则的索引顺序表，第一条分类规则的索引号为 0

返回值：

- [HoloWANReturn](#) 对象。

注意： 在排序时需要将每一条规则的索引都填写到 list[] 中。若某条规则的索引没有填写在 list[] 中，则该规则会被删除。
例如容器内有 4 条分类规则 a, b, c, d，索引为 0, 1, 2, 3，传入的 order 列表为[0,3,1]，重排后的容器内规则为：a, d, b。分类规则 c 被删除。

4.3 分类规则

报文分类器的分类规则。

4.3.1 IPv4 地址分类规则（IPv4Rule）

IPv4 地址分类规则，通过填写报文的源 IPv4 地址和目标 IPv4 地址来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：


```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv4Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 IPv4 地址分类规则参数
ipv4 = IPv4Rule(src="any", smask=32, dst="any", dmask=32, tos="any", action=1)
# 将 IPv4 地址分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(ipv4,port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str	源 IPv4 地址，可以填写：单一的IPv4 地址；由“IPv4地址-IPv4地址”指定一个范围；通配符“any”匹配任意的IPv4 地址。
smask	int	源 IPv4 地址掩码，填写范围 1-32。掩码为 24 则表示只匹配 IPv4 地址的前 24 个bit。
dst	str	目标 IPv4 地址，可以填写：单一的IPv4 地址；由“IPv4地址-IPv4地址”指定一个范围；通配符“any”匹配任意的IPv4 地址。
dmask	int	目标 IPv4 地址掩码，填写范围 1-32。掩码为 24 则表示只匹配 IPv4 地址的前 24 个bit。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.2 IPv6 地址分类规则（IPv6Rule）

IPv6 地址分类规则，通过填写报文的源 IPv6 地址和目标 IPv6 地址来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv6Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 IPv6 地址分类规则参数
ipv6 = IPv6Rule(src="any", dst="any", action=1)
# 将 IPv6 地址分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(ipv6,port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str	源 IPv6 地址，可以填写：单一的 IPv6 地址；通配符“any”匹配任意的 IPv6 地址。
dst	str	目标 IPv6 地址，可以填写：单一的 IPv6 地址；通配符“any”匹配任意的 IPv6 地址。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.3 MAC 地址分类规则（MACRule）

MAC 地址分类规则，通过填写报文的源 MAC 地址和目标 MAC 地址来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import IPv6Rule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 MAC 地址分类规则参数
mac = MACRule(src="any", dst="any", type="any", action=1)
# 将 MAC 地址分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(mac,port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str	源 MAC 地址，可以填写：单一的 MAC 地址；通配符“any”匹配任意的 MAC 地址。
dst	str	目标 MAC 地址，可以填写：单一的 MAC 地址；通配符“any”匹配任意的 MAC 地址。
type	str	在符合 MAC 地址的规则的基础上筛选出指定类型的报文，如不需分类则选择“any”。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.4 MPLS 分类规则（MPLSRule）

MPLS 分类规则，通过填写报文的 MPLS 标签来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine import Engine
from holowan.v2.engine.classifier import MPLSRule

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 MPLS 分类规则参数
mpls = MPLSRule(label="any", action=1)
# 将 MPLS 分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(mpls, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
label	str	MPLS 标签，可以填写：单一的 MPLS 标签；通配符“any”匹配任意的 MPLS 报文。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.5 PPPoE 分类规则（PPPoERule）

PPPoE 分类规则，通过填写报文的 Session ID 和 Code 来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import PPPoERule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 PPPoE 分类规则参数
pppoe = PPPoERule(sid="any", code="any", action=1)
# 将 PPPoE 分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(pppoe, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
sid	str	报文的 Session ID，可以填写：单一的 SID；通配符“any”匹配任意的 PPPoE 报文。
code	str	报文的 CODE，可以填写：单一的 CODE；通配符“any”匹配任意的 PPPoE 报文。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表 path2。

属性：

- **action:** int. 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.6 字节偏移分类规则（RawByteRule）

字节偏移分类规则，通过报文中特定的某一个或多个字节来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import RawByteRule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 定义 Raw 1-Byte 字节偏移分类规则方法
raw1 = RawByteRule(type=1, action=1)
# 设置 Raw 1-Byte 字节偏移分类规则具体参数
raw1.add_raw_byte(layer=2, offset=0, mask="0xFF", value="0x11")

# 定义 Raw 4-Byte 字节偏移分类规则方法
raw2 = RawByteRule(type=4, action=1)
# 设置 Raw 4-Byte 字节偏移分类规则具体参数
raw2.add_raw_byte(layer=2, offset=0, mask="0xFF00FF00", value="0x11223344")

# 将 Raw 1-Byte 字节偏移分类规则应用到 port1 上
result1 = engine1.apply_rule_to_classifier(raw1, port=1)
# 将 Raw 4-Byte 字节偏移分类规则应用到 port2 上
result2 = engine1.apply_rule_to_classifier(raw2, port=2)

# 打印 result 判断是否应用成功
print(result1, result2)
```

参数：

参数	类型	参数含义
type	int	分类规则的类型，1 表示 RAW 1-Byte Offsets 分类规则，匹配报文中的某一个字节；4 表示 Raw 4-Byte Offsets 分类规则，匹配报文中的某四个字节。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。
layer	int	偏移类型，指定是在报文中对字节进行计数的起始位置。2 表示从 Ether、VLAN 等二层协议的头部开始计数，3 表示从 IPv4、IPv6、MPLS、PPPoE 等三层协议的头部开始计数。
offset	int	目标字节的偏移位置。若 layer=2，则 offset 范围为 0-31；layer=3，则 offset 范围为 0-1966。
mask	str	掩码，标识目标字节中待匹配的比特。采用 16 进制，范围为 0x00-0xFF。
value	str	目标字节的值，采用 16 进制，范围为 0x00-0xFF。

属性：

- **action:** int类型 可读可写。

方法：

- `add_raw_byte(layer: int, offset: int, mask: str, value: str)` 可以添加一条 raw byte offset。

返回值：

- `HoloWANReturn` 对象。

4.3.7 TCP 端口号分类规则（TCPRule）

TCP 端口号分类规则，通过填写 TCP 报文的源端口号和目标端口号来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import TCPRule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 TCP 端口号分类规则参数
tcp = TCPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
              check_version=0, action=1)
# 将 TCP 端口号分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(tcp, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str / int / list[str/int]	TCP 报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
dst	str / int / list[str/int]	TCP 报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.8 SCTP 端口号分类规则（SCTPRule）

SCTP 端口号分类规则，通过填写 SCTP 报文的源端口号和目标端口号来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import SCTPRule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 SCTP 端口号分类规则参数
sctp = SCTPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
                check_version=0, action=1)
# 将 SCTP 端口号分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(sctp, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str / int / list[str/int]	SCTP 报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
dst	str / int / list[str/int]	SCTP 报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.9 UDP 端口号分类规则（UDPRule）

UDP 端口号分类规则，通过填写 UDP 报文的源端口号和目标端口号来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import UDPRule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 UDP 端口号分类规则参数
udp = UDPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
              check_version=0, action=1)

# 将 UDP 端口号分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(udp, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
src	str / int / list[str/int]	UDP 报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
dst	str / int / list[str/int]	UDP 报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符“any”匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.10 VLAN 分类规则（VLANRule）

VLAN 分类规则，通过填写报文 VLAN tag 的 PCP 值和 VLAN ID 来匹配特定的报文。并将匹配到的报文按照 action 中的动作进行下一步操作。

举例：

```
from holowan.v2.engine.classifier import VLANRule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置 VLAN 分类规则参数
vlan = VLANRule(fpcp="any", fpid="any", action=1)
vlan.enable_second_tag(spcp="any", spid="any")
# 将 VLAN 分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(vlan, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
fpcp	str	VLAN 中用于确定优先级的字段，可以填写：0-7 中的任意一个数字；通配符“any”匹配所有的 PCP 字段。
fpid	str	VLAN ID，可以填写：1-4094 中的任意一个数字；通配符“any”匹配所有的 VLAN ID。
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表 path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.11 组合分类规则（CombinationRule）

组合分类规则可以将 MAC 地址分类规则、IPv4 地址分类规则、IPv6 地址分类规则、VLAN 分类规则、MPLS 分类规则、TCP/UDP/SCTP 分类规则组合使用，组合的规则之间是与的关系。匹配到的报文将按照 action 中的动作进行

下一步操作。

举例：

```
from holowan.v2.engine.classifier import CombinationRule, TCPRule, MACRule, PPPoERule
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 设置组合分类规则参数
comb = CombinationRule(action=1)
comb.add_rule(
    TCPRule(src=123, dst=123, check_version=0, action=1),
    MACRule(src="any", dst="any", type="any", action=1),
    PPPoERule(sid="any", code="any", action=1)
)
# 将组合分类规则应用到 port1 上
result = engine1.apply_rule_to_classifier(comb, port=1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
action	int	被此规则匹配的报文将如何操作，-1：报文将被透传（Bypass）不受任何损伤；-2：报文将被丢弃（Drop）；1-15：报文将被转发到指定的 Path 上，1 代表 path1，2 代表path2。

属性：

- **action:** int类型 可读可写。

返回值：

- [HoloWANReturn](#) 对象。

4.3.12 设置分类规则的名称 set_custom_name(name:str)

可以为分类规则设置自定义的名称。如果未设置分类规则的名称，分类规则将使用默认的名称。

举例：

```

from holowan.v2.engine import Engine # 导入引擎模块
from holowan.v2.engine.classifier import SCTPRule, MACRule, IPv4Rule

holowan_ip = "192.168.1.199"      # 获取待操作的引擎
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)

mac = MACRule(src="any", dst="any", type="any", action=1)
mac.set_custom_name("my mac rule 1")      # 设置规则名称

vlan = VLANRule(fpcp="any", fpid="any", action=1)
vlan.set_custom_name("my vlan rule 1")     # 设置规则名称

result1 = engine1.apply_rule_to_classifier(mac, port=1)
print(result1)

result1 = engine1.apply_rule_to_classifier(vlan, port=1)
print(result1)

```

参数：

参数	类型	参数含义
name	str	待设置的名称

5. 虚拟链路 Path

仿真引擎的虚拟链路 Path 类。在 Path 上，可以设置带宽限制、时延、丢包、误码等损伤。

举例：

```
from holowan.v2.engine.path import Path
from holowan.v2.engine.path import BandwidthFixed
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip, holowan_port, engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 port1 到 port2 方向上的带宽限制
path1.l2r.bandwidth = BandwidthFixed(rate=100, unit=2)
# 应用设置的损伤
result = engine.apply_path_configuration(path1)

print(result) # 打印 result 判断是否应用成功
print(path1) # 打印 path1 上的损伤参数信息
```

注意： Path 类自身不具备将配置应用到 HoloWAN 的能力，也不具备从 HoloWAN 获取最新配置的能力。

5.1 属性

每条虚拟链路都有两个方向，分别是 port1 -> port2、port2 -> port1 方向。这两个方向对应为 Path 类的两个属性：l2r 和 r2l。

- **l2r**：Impairments 类实例对象。虚拟链路 port1 -> port2 方向的损伤集合。
- **r2l**：Impairments 类实例对象。虚拟链路 port2 -> port1 方向的损伤集合。
- **path_id**：int 类型 虚拟链路的 ID。
- **path_name**：int 类型 虚拟链路的名称。
- **engine_id**：int 类型 虚拟链路所在的 Engine 的 ID。

5.2 方法

5.2.1 重置链路 Path.reset()

将虚拟链路两个方向的损伤配置重置为初始状态。

举例：

```
from holowan.v2.engine.path import Path
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

path1 = engine.get_path_by_id(path_id) # 获取 path1
path1.reset() # 重置 path1
result = engine.apply_path_configuration(path1)

print(result) # 打印 result 判断是否应用成功
```

如果您只需要重置某条虚拟链路，可以直接使用Engine.reset_path()方法。

返回值：

- [HoloWANReturn](#) 对象。

5.2.2 更新链路 Path.update()

该方法不具备将配置应用到 HoloWAN 的能力，也不具备从 HoloWAN 获取最新配置的能力。该方法只更新其 XML。

5.3 损伤类型 Impairment

HoloWAN 的损伤类型。

5.3.1 带宽限制 (Bandwidth)

5.3.1.1 固定带宽 (Fixed)

带宽限制：固定带宽模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BandwidthFixed
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的带宽限制
path1.l2r.bandwidth = BandwidthFixed(rate=999.9, unit=1)
# 应用损伤参数
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
rate	float	带宽限制的数值，允许填写小数点后 1 位，单位为bps时不允许填写小数。
unit	int	带宽限制的单位，1： bps， 2： Kbps， 3： Mbps。

属性：

- **rate:** float 类型. 带宽的数值。可读可写。
- **unit:** int 类型. 带宽的单位。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.1.2 抖动模式（BandwidthJitter）

带宽限制：抖动模式。所需参数与 HoloWAN Web GUI 界面一致。不同曲线模型需要填写的参数不同，以前端界面参数为准。

举例：

```
from holowan.v2.engine.path import BandwidthJitter
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的带宽限制
path1.l2r.bandwidth = BandwidthJitter(unit=1, change_mode=ChangeMode(
                                mode=1, max=99.9, min=11.1, phase=60, period=100)

# 应用损伤参数
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
unit	int	带宽限制的单位，1： bps， 2： Kbps， 3： Mbps。
change_mode	ChangeMode	实例对象。带宽抖动的变化模式。

1.正弦曲线

参数	类型	参数含义
mode	int	mode = 1。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
phase	int	曲线的初始位置，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

2.峰型曲线

参数	类型	参数含义
mode	int	mode = 2。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写至小数点后 1 位，单位为bps时不允许填写小数。

参数	类型	参数含义
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
rise	int	曲线的上升部分占曲线的比例，填写范围 0-100。
fall	int	曲线的下降部分占曲线的比例，填写范围 0-100
period	int	曲线变化的周期，填写范围 1-3600。

3.倒钟型曲线

参数	类型	参数含义
mode	int	mode = 3。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
period	int	曲线变化的周期，填写范围 1-3600。

4.钟型曲线

参数	类型	参数含义
mode	int	mode = 4。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
period	int	曲线变化的周期，填写范围 1-3600。

5.反Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 5。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写小数点后 1 位，单位为bps时不允许填写小数。
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写小数点后 1 位，单位为bps时不允许填写小数。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

6.Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 6。
max	float	带宽变化的最大值，填写范围 1-1000，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
min	float	带宽变化的最小值，填写范围 1-999.9，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

属性：

- **unit:** int类型 带宽的单位。可读可写。
- **change_mode:** ChangeMode 实例对象。带宽抖动的变化模式。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.1.3 令牌桶模式（TokenBucket）

带宽限制：令牌桶模式。所需参数与 HoloWAN Web GUI 界面一致。
共有三种令牌桶模式。

举例：

```
from holowan.v2.engine.path import BandwidthTokenBucket
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的带宽限制
path1.l2r.bandwidth = BandwidthTokenBucket(type=3,cir=10,cir_unit=3,cbs=10,cbs_unit=3,
                                             pir=10,pir_unit=3,pbs=50,pbs_unit=3)

# 应用损伤参数
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

1.单令牌桶模式 Single Bucket

参数	类型	参数含义
type	int	单令牌桶模式 type = 1。
rate	float	报文发送速率，对应令牌的添加速率，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
rate_unit	int	报文发送速率的单位，1： bps， 2： Kbps， 3： Mbps。
burst	float	允许突发的最高速率，对应令牌桶的容量大小，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
burst_unit	int	突发的最高速率的单位，1： bps， 2： Kbps， 3： Mbps。

2. 单速率双令牌桶模式 Double Bucket(srTCM)

参数	类型	参数含义
type	int	单速率双令牌桶模式模式 type = 2。
cir	float	承诺信息速率，即令牌的添加速率，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
cir_unit	int	cir 的单位，1： bps， 2： Kbps， 3： Mbps。
cbs	float	承诺突发尺寸，即 C 桶的容量大小，填写范围 0-1000，允许填写至小数点后 1 位。
cbs_unit	int	cbs的单位，1： B， 2： KB， 3： MB。
ebs	float	超额突发尺寸，即 E 桶的容量大小，填写范围 0-1000，允许填写至小数点后 1 位。
ebs_unit	int	ebs的单位，1： B， 2： KB， 3： MB。

3. 双速率双令牌桶模式 Double Bucket(trTCM)

参数	类型	参数含义
type	int	双速率双令牌桶模式 type = 3。
cir	float	承诺信息速率，即 C 桶令牌的添加速率，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
cir_unit	int	cir 的单位，1： bps， 2： Kbps， 3： Mbps。
cbs	float	承诺突发尺寸，即 C 桶的容量大小，填写范围 0-1000，允许填写至小数点后 1 位。
cbs_unit	int	cbs 的单位，1： B， 2： KB， 3： MB。
pir	float	峰值信息速率，即 P 桶令牌的添加速率，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
pir_unit	int	pir 的单位，1： bps， 2： Kbps， 3： Mbps。

参数	类型	参数含义
pbs	float	峰值突发尺寸，即 P 桶的容量大小，填写范围 0-1000，允许填写至小数点后 1 位。
pbs_unit	int	pbs 的单位，1： B， 2： KB， 3： MB。

返回值：

- [HoloWANReturn](#) 对象。

5.3.1.4 双向带宽限制模式（Bidirectional）

带宽限制：双向带宽限制模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BandwidthBidirectional
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的带宽限制
path1.l2r.bandwidth = BandwidthBidirectional(rate=10, unit=3)
# 应用损伤参数
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
rate	float	双向带宽限制大小，允许填写至小数点后 1 位，单位为bps时不允许填写小数。
unit	int	报文发送速率的单位，1： bps， 2： Kbps， 3： Mbps。

返回值：

- [HoloWANReturn](#) 对象。

5.3.2 背景流量（BackgroundUtilization）

5.3.2.1 禁用背景流量（Disable）

禁用背景流量。

举例：

```
from holowan.v2.engine.path import BackgroundUtilizationDisable
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向禁用背景流量
path1.l2r.background_utilization = BackgroundUtilizationDisable()
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

5.3.2.2 随机模式（Random）

背景流量：随机模式，所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BackgroundUtilizationRandom
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的背景流量参数
path1.l2r.background_utilization = BackgroundUtilizationRandom(rate=10,burst=60)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
rate	float	背景流量的产生速率占带宽限制的比例。填写范围 0-100，允许填写至小数点后一位。
burst	int	背景流量的帧大小。填写范围 64 - 10000。

属性：

- **rate:** float类型，背景流量的产生速率。可读可写。
- **burst:** int类型，背景流量的帧大小。带宽抖动的变化模式。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.2.3 PCAP 模式（PCAP）

背景流量：PCAP 模式，所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BackgroundUtilizationPCAP
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的背景流量参数
path1.l2r.background_utilization = BackgroundUtilizationPCAP(rate=10, pcap_name="mlx.pcap")
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
rate	float	背景流量的产生速率占带宽限制的比例。填写范围 0-100，允许填写至小数点后一位。
pcap_name	string	pcap文件的名称。

返回值：

- [HoloWANReturn](#) 对象。
-

5.3.3 队列深度（QueueLimit）

5.3.3.1 简易模式（Simple）

队列深度：简单模式。简易模式提供了极小、小、适中、大 四种模式的队列深度。

举例：

```
from holowan.v2.engine.path import QueueLimitSimple
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的队列深度
path1.l2r.queue_limit = QueueLimitSimple(option=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
option	int	4 种模式的队列深度。1：极小 16KB，2：小 256KB，3：适中 250ms，4：大 256MB

5.3.3.2 尾部丢弃模式（DropTail）

队列深度：尾部丢弃模式，队列已满时所有到达 HoloWAN 的报文都被丢弃。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import QueueLimitDropTail
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的队列深度
path1.l2r.queue_limit = QueueLimitDropTail(depth=300, unit=2)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
depth	int	队列深度值，单位不同时填写范围不同。Packets：0-1638400，KB：0-104857600，ms：0-1000。
unit	int	队列深度的单位。1：Packets，2：KB，3：ms。

属性：

- **depth**：int类型，队列深度值。可读可写。
- **unit**：int类型，队列深度的单位。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.3.3 RED模式（RED）

队列深度：随机早检测模式，队列快要满时，对即将进入队列的报文随机进行丢弃。所需参数与 HoloWAN Web GUI 界面一致。

举例：


```
from holowan.v2.engine.path import QueueLimitRED
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的队列深度
path1.l2r.queue_limit = QueueLimitRED(weight=0.001, min_threshold=8,
                                       max_threshold=21, probability=0.3)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
weight	float	权值，填写范围 0-1，允许填写至小数点后三位。
min_threshold	int	最小队列长度阈值，填写范围 0-102400。
max_threshold	int	最大队列长度阈值，填写范围 0-102400。
probability	float	最大丢包概率，填写范围 0-1，允许填写至小数点后三位。

属性：

- **weight**： float类型，权值。可读可写。
- **min_threshold**： int类型，最小队列长度阈值。可读可写。
- **max_threshold**： int类型，最大队列长度阈值。可读可写。
- **probability**： float类型，最大丢包概率。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.4 帧开销（FrameOverhead）

5.3.4.1 24 字节模式（24-Ethernet）

帧开销：24-Ethernet模式，标准的以太网帧开销固定为 24 字节。

举例：

```

from holowan.v2.engine.path import FrameOverhead24Ethernet
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的帧开销的模式
path1.l2r.frame_overhead = FrameOverhead24Ethernet()
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

5.3.4.2 4 字节模式（4-Ethernet）

帧开销：4-Ethernet模式，该模式下的帧开销固定为 4 字节。

举例：

```

from holowan.v2.engine.path import FrameOverhead4Ethernet
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的帧开销的模式
path1.l2r.frame_overhead = FrameOverhead4Ethernet()
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

5.3.4.3 自定义模式（Custom）

帧开销：自定义模式，帧开销的字节数可自定义。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import FrameOverheadCustom
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向的帧开销
path1.l2r.frame_overhead = FrameOverheadCustom(size=64)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
size	int	帧开销大小，填写范围 0-64

属性：

- **size**：帧开销大小。可读可写。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5 报文修改（Modify）

报文修改可以将报文中指定的字节修改为其他值。报文修改的每个模式都提供了两种传参形式。

5.3.5.1 禁用报文修改（Disable）

禁用报文修改。

举例：

```
from holowan.v2.engine.path import ModifyDisable
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 to port2 方向禁用报文修改
path1.l2r.modify = ModifyDisable()
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.2 全部修改模式（Normal）

报文修改：全部修改模式，符合 Match 规则的报文被全部修改。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import ModifyNormal
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyNormal(match_header=1, match_offset=0, match_size=1,
                                match_value="0x0F", modify_header=1,
                                modify_offset=0, modify_size=1, modify_value="0xFF",
                                crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x00",
}
modify_params = {
    "modify_header":1, "modify_offset":0, "modify_size":1, "modify_value":"0x0F"
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyNormal(match_params = match_params,crc=1,
                                modify_params=modify_params,checksum=1)

# 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。

参数	类型	参数含义
modify_header	int	报文修改时，Offset的相对起始位置。
modify_offset	int	指定一定的偏移量，确定需要进行修改的字节位置。
modify_size	int	指定需要修改的字节数，填写范围1、2、4、6、8。
modify_value	str	指定修改为何值，填写范围 00-FF。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含："match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
modify_params	dict	报文修改的参数字典。包含："modify_header", "modify_offset", "modify_size", "modify_value"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.3 周期修改（Cycle）

报文修改：周期修改模式，符合 Match 规则的报文被周期性修改。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import ModifyCycle
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyCycle(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                modify_header=1,modify_offset=0, modify_size=1,
                                modify_value="0xF", cycle_period=1000, cycle_burst=100,
                                crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
modify_params = {
    "modify_header":1, "modify_offset":0, "modify_size":1, "modify_value":"0xF",
    "cycle_period":1000, "cycle_burst":100,
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyCycle(match_params = match_params,modify_params=modify_params,
                                crc=1, checksum=1)

# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。

参数	类型	参数含义
modify_header	int	报文修改时，Offset的相对起始位置。
modify_offset	int	指定一定的偏移量，确定需要进行修改的字节位置。
modify_size	int	指定需要修改的字节数，填写范围1、2、4、6、8。
modify_value	str	指定修改为何值，填写范围 00-FF。
cycle_period	int	报文修改的周期，填写范围 1-99999999。
cycle_burst	int	每个周期内被修改的报文数量，填写范围 0-99999998。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含："match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
modify_params	dict	报文修改的参数字典。包含："modify_header", "modify_offset", "modify_size", "modify_value", "cycle_period", "cycle_burst"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.4 固定概率修改（Random）

报文修改：固定概率修改模式，符合 Match 规则的报文按照固定的概率被修改。所需参数与 HoloWAN Web GUI 界面一致。

举例：


```

from holowan.v2.engine.path import ModifyRandom
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyRandom(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                modify_header=1,modify_offset=0, modify_size=1,
                                modify_value="0xF", random_rate=0.001,crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
modify_params = {
    "modify_header":1, "modify_offset":0, "modify_size":1, "modify_value":"0xF",
    "random_rate":0.1,
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyRandom(match_params = match_params,modify_params=modify_params,
                                crc=1, checksum=1)

# 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。

参数	类型	参数含义
modify_header	int	报文修改时，Offset的相对起始位置。
modify_offset	int	指定一定的偏移量，确定需要进行修改的字节位置。
modify_size	int	指定需要修改的字节数，填写范围1、2、4、6、8。
modify_value	str	指定修改为何值，填写范围 00-FF。
random_rate	float	报文修改的概率，填写范围 0-100。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含： "match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
modify_params	dict	报文修改的参数字典。包含： "modify_header", "modify_offset", "modify_size", "modify_value", "random_rate"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.5 多范围模式（Range）

报文修改：多范围修改模式，可以修改符合 Match 规则的报文中多个不相邻的字节。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import ModifyRanges
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
# 设置报文修改范围
modify_ranges = [
    {"offset": 0, "value": "aa,bb"},
    {"offset": 10, "value": "ee"}]
# 设置 path1 上 port1 到 port2 方向的报文修改
path1.l2r.modify = ModifyRanges(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                modify_ranges=modify_ranges, crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
modify_ranges = [
    {"offset": 0, "value": "aa,bb"},
    {"offset": 10, "value": "ee"}]
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyRanges(match_params = match_params,modify_ranges=modify_ranges,
                                crc=1, checksum=1)

# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。

参数	类型	参数含义
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。
modify_ranges	list	offset 是报文修改时的相对起始位置；range指定将报文修改为何值，填写范围 00-FF，最多可填写10个字节。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含："match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
modify_ranges	list	offset 是报文修改时的相对起始位置；range指定将报文修改为何值，填写范围 00-FF，最多可填写10个字节。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.6 插入模式（Insert）

报文修改：插入模式，在符合 Match 规则的报文中插入字节。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import ModifyInsert
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyInsert(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                insert_offset=1, insert_value="aa,bb,cc", crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
insert_params = {
    "insert_offset": 10, "insert_value": "ff"
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyInsert(match_params=match_params, insert_params=insert_params, crc=1, che
# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。
insert_offset	int	指定一定的偏移量，确定需要插入的字节位置。
insert_value	str	指定插入的字节值，最多插入 10 个字节，填写范围 00-FF。

参数	类型	参数含义
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含： "match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
insert_params	dict	报文插入的参数字典。包含： "insert_offset", "insert_value"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.7 交换模式（Exchange）

报文修改：交换模式，可以将目标报文中的两个字节范围进行交换。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import ModifyExchange
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyExchange(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                   previous_offset=12, previous_size=21, next_offset=34, next_size=43, crc=1, checksum=0)
# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
exchange_params = {
    "previous_offset": 10, "previous_size": 10, "next_offset": 30, "next_size": 53
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyExchange(match_params=match_params, exchange_params=exchange_params, crc=1, checksum=0)
# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)
```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。
match_value	str	所要匹配的值，填写范围 00-FF。
previous_offset	int	前一个字节范围的第一个字节相较于报文头的偏移量。
previous_size	int	前一个字节范围的长度。

参数	类型	参数含义
next_offset	int	后一个字节范围的第一个字节相较于报文头的偏移量。
next_size	int	前一个字节范围的长度。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含： "match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
exchange_params	dict	报文交换的参数字典。包含： "previous_offset", "previous_size", "next_offset", "next_size"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.8 次数修改模式（Count）

报文修改：次数修改模式，可指定连续修改若干个报文。所需参数与 HoloWAN Web GUI 界面一致。

举例：


```

from holowan.v2.engine.path import ModifyCount
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
# 设置报文修改范围
modify_ranges = [
    {"offset": 0, "value": "aa,bb"},
    {"offset": 10, "value": "ee"}]
# 设置 path1 上 port1 to port2 方向的报文修改
path1.l2r.modify = ModifyCount(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                offset=12, value="ff", modify_offset=34, modify_count=43, crc=1, checksum=)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
count_params = {
    "offset": 22, "value": "ee", "modify_offset": 44, "modify_count": 53
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyCount(match_params=match_params, count_params=count_params, crc=1, checksum=)
# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。

参数	类型	参数含义
match_value	str	所要匹配的值，填写范围 00-FF。
offset	int	被修改的目标字节相较于报文头的偏移量
value	str	修改的目标值，填写范围 00-FF。
modify_offset	int	指定从第几个报文修改。
modify_count	int	指定要修改的报文数量。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含： "match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
count_params	list	报文修改的参数字典。包含： "offset", "value", "modify_offset", "modify_count"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.9 删除模式（Delete）

报文修改：删除模式，删除报文中的某些字节。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import ModifyDelete
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 配置报文修改的参数，参数设置有 2 种方式，选择下述任意一种即可
# 形式1 直接传参，设置 path1 上 port1 到 port2 方向上的报文修改
# 设置报文修改范围
modify_ranges = [
    {"offset": 0, "value": "aa,bb"},
    {"offset": 10, "value": "ee"}]
# 设置 path1 上 port1 to port2 方向的报文修改
path1.l2r.modify = ModifyDelete(match_header=1, match_offset=0, match_size=1, match_value="0x0",
                                delete_offset=10, delete_size=5, crc=1, checksum=1)

# 形式1 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

# 形式2 通过参数字典形式传参
match_params = {
    "match_header":1, "match_offset":0, "match_size":1, "match_value":"0x0",
}
delete_params = {
    "delete_offset": 20, "delete_size": 8
}
# 形式2 设置 path1 上 port1 到 port2 方向上的报文修改
path1.l2r.modify = ModifyDelete(match_params=match_params, delete_params=delete_params, crc=1, che
# 形式2 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)

```

参数：

形式1

参数	类型	参数含义
match_header	int	报文匹配时，Offset 的相对起始位置。
match_offset	int	指定一定的偏移量，确定进行规则匹配的起始位置，填写范围 0-1493。
match_size	int	指定需要匹配的字节数，填写范围1、2、4、6、8。

参数	类型	参数含义
match_value	str	所要匹配的值，填写范围 00-FF。
delete_offset	int	被删除的字节距离报文头的偏移量。
delete_size	int	被删除的字节长度。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

形式2

参数	类型	参数含义
match_params	dict	报文匹配的参数字典。包含： "match_header", "match_offset", "match_size", "match_value"。参数填写范围参照形式1。
delete_params	dict	报文修改的参数字典。包含： "delete_offset", "delete_size"。参数填写范围参照形式1。
checksum	int	被修改的报文是否会重新计算checksum。1 为是，0 为否。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.6 大包分片（MTU）

5.3.5.1 禁用大包分片（Disable）

禁用大包分片。

举例：

```

from holowan.v2.engine.path import MTUDisable
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 禁用 path1 上 port1 到 port2 方向上的 MTU
path1.l2r.mtu = MTUDisable()
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

返回值：

- [HoloWANReturn](#) 对象。

5.3.5.2 大包分片（MTULimit）

大包分片，大于 MTU 值的报文将被分片。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import MTULimit
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的 MTU
path1.l2r.mtu = MTULimit(limit=1499, drop_df_packets=1, drop_oversize_packets=0)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

参数：

参数	类型	参数含义
limit	int	分片数值，大于该数值的数据包将被分片，填写范围 68 - 9214。
drop_df_packets	int	大于 MTU 数值的数据包将被直接丢弃。1为是，0为否。
drop_oversize_packets	int	不允许被分片的数据包将被直接丢弃。1为是，0为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7 时延（Delay）

5.3.7.1 常量模式（Constant）

时延：常量模式，所有报文被添加相同的固定时延。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayConstant
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的时延
path1.l2r.delay = DelayConstant(delay=99.9)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
delay	float	报文受到的时延，填写范围 0-10000，允许填写至小数点后两位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.2 均匀分布（Uniform）

时延：均匀分布模式，报文将受到均匀分布的时延。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayUniform
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)

# 设置 path1 上 port1 到 port2 方向上的均匀分布时延参数
path1.l2r.delay = DelayUniform(minimum=11.1, maximum=99.9,
                                enable_reordering=1,advanced_setup=None)

# 应用设置
result = engine.apply_path_configuration(path1)
# 打印 result 判断是否应用成功
print(result)
```

```
# 均匀分的高级模式
from holowan.v2.engine.path import DelayUniform, ChangeMode
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置均匀分布的参数
uniform = DelayUniform(minimum=11.1, maximum=99.9, enable_reordering=1,advanced_setup=None)
# 设置均匀分布高级模式的参数
uniform.set_advanced_setup(ChangeMode(mode=6, max=99.9, min=11.1, section=10, period=100))
# 将上述参数应用到 path1 的 port1 到 port2 方向上
path1.l2r.delay = uniform
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
minimum	float	时延的最小值，填写范围 0-9999.99，允许填写至小数点后 2 位。

参数	类型	参数含义
maximum	float	时延的最大值，填写范围 0.01-10000，允许填写至小数点后 2 位。
enable_reordering	int	是否允许时延抖动产生报文乱序。1为是，0为否。
advanced_setup	ChangeMode	实例对象。 高级设置可以在原有的均匀分布时延上叠加一个变化的时延，叠加的时延由6种曲线模型控制。

方法：

1. DelayUniform.set_advanced_setup(change_mode: ChangeMode)

设置均匀分布模式的高级设置。所需参数为 ChangeMode 类的实例对象。

2. DelayUniform.disable_advanced_setup()

禁用高级设置。

1.正弦曲线

参数	类型	参数含义
mode	int	mode = 1。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
phase	int	曲线的初始位置，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

2.峰型曲线

参数	类型	参数含义
mode	int	mode = 2。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
rise	int	曲线的上升部分占曲线的比例，填写范围 0-100。
fall	int	曲线的下降部分占曲线的比例，填写范围 0-100
period	int	曲线变化的周期，填写范围 1-3600。

3.倒钟型曲线

参数	类型	参数含义
mode	int	mode = 3。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。

参数	类型	参数含义
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
period	int	曲线变化的周期，填写范围 1-3600。

4.钟型曲线

参数	类型	参数含义
mode	int	mode = 4。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
period	int	曲线变化的周期，填写范围 1-3600。

5.反Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 5。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

6.Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 6。
max	float	时延变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	时延变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.3 正态分布（Normal）

时延：正态分布模式，报文将受到正态分布的时延。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```

from holowan.v2.engine.path import DelayNormal
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的正态分布时延参数
path1.l2r.delay = DelayNormal(minimum=1.11, mean=55.55, std_deviation=11.11,
                               enable_reordering=1)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

```

# 正态分布的高级模式
from holowan.v2.engine.path import DelayUniform, ChangeMode
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置正态分布的参数
normal = DelayNormal(minimum=1.11, mean=55.55, std_deviation=11.11, enable_reordering=1)
# 设置正态分布高级模式参数
normal.enable_advanced_setup(period=50, min=900.1, max=1000, duration=2)
# 将上述参数应用到 path1 的 port1 到 port2 方向上
path1.l2r.delay = normal
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)

```

参数：

参数	类型	参数含义
minimum	float	时延的最小值，填写范围 0-mean，允许填写至小数点后 2 位。

参数	类型	参数含义
mean	float	时延的平均值，填写范围 minimum&std_deviation < mean < 10000-std_deviation，允许填写至小数点后 2 位。
std_deviation	float	标准差，填写范围 std_deviation < mean，允许填写至小数点后 2 位。
enable_reordering	int	是否允许时延抖动产生报文乱序。1为是，0为否。

方法：

1. DelayNormal.enable_advanced_setup(period: int, duration: int, min: float, max: float)
正态分布时延的高级设置。可以在正态分布时延过程中周期突发一个较大的时延。突发的时延与原有的正态分布时延不会叠加。必须以关键字形式传参。

2. DelayNormal.disable_advanced_setup()
禁用高级设置。

参数：

参数	类型	参数含义
period	int	突发时延的周期，填写范围 0-3600。
min	float	突发时延的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
max	float	突发时延的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
duration	int	突发时延的持续时间，填写范围 1-3599。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.4 自定义的正态分布（Custom）

时延：自定义的正态分布模式。所需参数与 HoloWAN Web GUI 界面一致。该模式下的参数相互影响，填写时请注意参数的数值是否符合要求。

举例：

```
from holowan.v2.engine.path import DelayCustom
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的自定义的正态分布的时延参数
path1.l2r.delay = DelayCustom(mean_delay=100.0, min_delay=1.0, max_delay=200.0,
                               pos_delta=30.0, neg_delta=30.0, spread=1.59,
                               enable_reordering=1)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
mean_delay	float	时延的均值，填写范围 min_delay < mean_delay < max_delay
min_delay	float	时延的最小值，填写范围 0 ~ （mean_delay - neg_delta）
max_delay	float	时延的最大值，填写范围 （mean_delay + pos_delta） ~ 10000
pos_delta	float	延迟变化的上限，填写范围 pos_delta < (max_delay - mean_delay)。
neg_delta	float	延迟变化的下限，填写范围 neg_delta < (mean_delay - min_delay)。
spread	float	弹性系数，填写范围 0.01-100。
enable_reordering	int	是否允许时延抖动产生报文乱序。1为是，0为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.5 抖动模式（Jitter）

时延：抖动模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayJitter
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的时延参数
path1.l2r.delay = DelayJitter(delay=99.9, jitter=51.1, type=1, enable_reordering=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
delay	float	固定的时延，填写范围 0-10000，允许填写至小数点后 2 位。
jitter	float	延迟的抖动值，填写范围 0-10000，允许填写至小数点后 2 位。
type	float	jitter 的抖动类型。1：固定值，2：平均分布，3：正态分布。
enable_reordering	int	是否允许时延抖动产生报文乱序。1为是，0为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.6 伽马分布（Gamma）

时延：Gamma分布模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayGamma
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的时延参数
path1.l2r.delay = DelayGamma(shape=1.1,scale=2.1,enable_reordering=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
shape	float	伽马分布曲线的形状，填写范围 0 <= shape*scale <= 10000，允许填写至小数点后 2 位。
scale	float	伽马分布的规模，填写范围 0 <= shape*scale <= 10000，允许填写至小数点后 2 位。
enable_reordering	int	是否允许时延抖动产生报文乱序。1为是，0为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.7 累积突发模式（AccumulateBurst）

时延：Accumulate & Burst 模式，报文进入 HoloWAN 后将不会立刻发送，而是会等待。直到累积报文个数或累积的时间达到设定的值后一次性发送出去，发送时会被额外添加一个固定的时延。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayAccumulateBurst
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的时延参数
path1.l2r.delay = DelayAccumulateBurst(delay_count=100, delay=99.9, extra_delay=50.1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数	类型	参数含义
delay_count	int	累积报文个数，填写范围 0-1000。
delay	float	报文累积的时间，填写范围 0-10000，允许填写至小数点后 2 位。
extra_delay	float	突发时延，填写范围 0-10000，允许填写至小数点后 2 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.7.8 自定义文件模式（Customized File）

时延：Accumulate & Burst 模式，报文进入 HoloWAN 后将不会立刻发送，而是会等待。直到累积报文个数或累积的时间达到设定的值后一次性发送出去，发送时会被额外添加一个固定的时延。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DelayCustomizedFile
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# Packet Mode模式，设置 path1 上 port1 到 port2 方向上的时延参数
path1.l2r.delay = DelayCustomizedFile(filename="1.txt", type=1, interval=100)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数	类型	参数含义
delay_count	int	累积报文个数，填写范围 0-1000。
filename	string	文件的名称，需填写已上传至 HoloWAN Delay 功能中的文件。
type	int	自定义文件功能模式，1：Packet Mode，2：Interval Mode
interval	int	时延变化的频率，type=1 时，该参数不生效，时延每个包变化一次；type=2时，可设置时延变化的频率，范围为 1-1000，单位ms。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8 丢包（Loss）

5.3.8.1 固定概率丢包（Random）

丢包：固定概率丢包，每个报文有一定的概率被丢弃。所需参数与 HoloWAN Web GUI 界面一致。

举例：


```
from holowan.v2.engine.path import LossRandom
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossRandom(loss_rate=0.001)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
loss_rate	float	每个报文丢弃的概率，填写范围 0-100，允许填写至小数点后 4 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.2 周期丢包（Cycle）

丢包：周期丢包，每个周期的最后若干个报文将被丢弃。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossCycle
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossCycle(period=900, burst=10)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
period	int	丢包的周期，填写范围 1-99999999。
burst	int	丢包的个数，填写范围 0-99999998。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.3 突发丢包（Burst）

丢包：突发丢包模式，设定每个报文被丢弃的概率，一旦丢包发生，则一次性丢弃若干的报文。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossBurst
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossBurst(probability=0.01, min=10, max=50)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
probability	int	每个报文被丢弃的概率，填写范围 0-100，允许填写至小数点后 4 位。
min	int	最小连续丢包数量，填写范围 0-99999999。
max	int	最大连续丢包数量，填写范围 0-99999999。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.4 双通道丢包（GilbertElliott）

丢包：双通道丢包模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossGilbertElliott
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossGilbertElliott(good_state_loss=0.1, good_state_change=0.2,
                                     bad_state_loss=0.3, bad_state_change=0.4)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
good_state_loss	float	良好通道丢包概率，填写范围 0-100，允许填写至小数点后 4 位。
good_state_change	float	良好通道转换到较差通道的概率，填写范围 0-100，允许填写至小数点后 4 位。
bad_state_loss	float	较差通道丢包概率，填写范围 0-100，允许填写至小数点后 4 位。
bad_state_change	float	较差通道转换到良好通道的概率，填写范围 0-100，允许填写至小数点后 4 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.5 四状态马尔科夫丢包（Markov）

丢包：四状态马尔科夫模式。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossMarkov
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossMarkov(p13=0.008, p31=0.05, p32=0.00001, p23=1, p14=0.0007)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
p13	float	状态 1 转换到 3 的概率，填写范围 0-100，允许填写至小数点后 3 位。
p31	float	状态 3 转换到 1 的概率，填写范围 0-100，允许填写至小数点后 3 位。
p32	float	状态 3 转换到 2 的概率，填写范围 0-100，允许填写至小数点后 3 位。
p23	float	状态 2 转换到 3 的概率，填写范围 0-100，允许填写至小数点后 3 位。
p14	float	状态 1 转换到 4 的概率，填写范围 0-100，允许填写至小数点后 3 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.6 曲线控制丢包（Jitter）

丢包：曲线控制丢包模式，可以通过不同的曲线模式控制丢包率的变化。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossJitter, ChangeMode
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossJitter(change_mode=ChangeMode(mode=6, max=99.9, min=9.9,
                                                    section=50, period=100))

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
change_mode	ChangeMode	实例对象，通过 6 种曲线模型控制丢包率的变化

1.正弦曲线

参数	类型	参数含义
mode	int	mode = 1。
max	float	丢包率变化的最大值，填写范围 0-100，允许填写至小数点后 4 位。
min	float	丢包率变化的最小值，填写范围 0-100，允许填写至小数点后 4 位。
phase	int	曲线的初始位置，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

2.峰型曲线

参数	类型	参数含义
mode	int	mode = 2。
max	float	丢包率变化的最大值，填写范围 0-100，允许填写至小数点后 4 位。
min	float	丢包率变化的最小值，填写范围 0-100，允许填写至小数点后 4 位。
rise	int	曲线的上升部分占曲线的比例，填写范围 0-100。
fall	int	曲线的下降部分占曲线的比例，填写范围 0-100

参数	类型	参数含义
period	int	曲线变化的周期，填写范围 1-3600。

3.倒钟型曲线

参数	类型	参数含义
mode	int	mode = 3。
max	float	丢包率变化的最大值，填写范围 0-100，允许填写至小数点后 4 位。
min	float	丢包率变化的最小值，填写范围 0-100，允许填写至小数点后 4 位。
period	int	曲线变化的周期，填写范围 1-3600。

4.钟型曲线

参数	类型	参数含义
mode	int	mode = 4。
max	float	丢包率变化的最大值，填写范围 0-100，允许填写至小数点后 4 位。
min	float	丢包率变化的最小值，填写范围 0-100，允许填写至小数点后 4 位。
period	int	曲线变化的周期，填写范围 1-3600。

5.反Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 5。
max	float	丢包率变化的最大值，填写范围 0-100，允许填写至小数点后 4 位。
min	float	丢包率变化的最小值，填写范围 0-100，允许填写至小数点后 4 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100，
period	int	曲线变化的周期，填写范围 1-3600。

6.Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 6。
max	float	丢包率变化的最大值，填写范围 0.1-10000，允许填写至小数点后 1 位。
min	float	丢包率变化的最小值，填写范围 0-9999.9，允许填写至小数点后 1 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

返回值：

- [HoloWANReturn](#) 对象。

5.3.8.7 次数丢包（Count）

丢包：次数丢包，设置从第n个包开始丢包m个包。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import LossCount
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的丢包参数
path1.l2r.loss = LossCount(start_offset=100, loss_count=10)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
start_offset	int	报文开始丢弃的位置，范围为 0-10000。
loss_count	int	丢包数量，范围为 0-10000。

返回值：

- [HoloWANReturn](#) 对象。

5.3.9 误码（Corruption）

5.3.9.1 全报文误码（BER）

误码：全报文误码模式，该模式基于报文的 bit 进行误码，误码概率采用科学计数法。所需参数与 HoloWAN Web GUI 界面一致。

举例：


```
from holowan.v2.engine.path import BER
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的误码参数
path1.l2r.corruption = BER(error_rate=3, error_rate_index=13, crc=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
error_rate	int	误码率的系数，填写范围 0-9。
error_rate_index	int	误码率的指数，填写范围 (3) ~ (14)。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.9.2 多范围误码（Range）

误码：多范围误码模式，可以在指定的字节范围内进行误码。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BERRange
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的误码参数
path1.l2r.corruption = BERRange(bit_error_rate=1, bit_error_rate_index=14,
                                crc=1, range_list=["1-10", "11-20", "21-30"])

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
bit_error_rate	int	误码率的系数，填写范围 0-9。
bit_error_rate_index	int	误码率的指数，填写范围 (3) ~ (14)。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。
range_list	list	误码范围的列表，列表中的后一个字节范围必须严格大于前一个字节范围。

返回值：

- [HoloWANReturn](#) 对象。

5.3.9.3 包错误（Packet）

误码：包错误模式，该模式基于报文(packet)进行误码，设置每个报文被误码的概率。一旦发生误码，报文中随机若干个 bit 将被修改。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BERPacket
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的误码参数
path1.l2r.corruption = BERPacket(ber_per_packet=10, probability=5, crc=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
ber_per_packet	int	每个报文发生的误码的比特数，填写范围 0-20。
probability	int	报文发生误码的概率，填写范围 0-100。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.9.4 次数误码（Bit Error Count）

误码：次数误码模式，设置误码开始的位置、误码报文数量、每个报文中错误的bit数量构造批量误码。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import BERCount
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的误码参数
path1.l2r.corruption = BERCount(ber_per_packet=10, start_offset=100, ber_count=5, crc=1)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
ber_per_packet	int	每个报文发生的误码的比特数，填写范围 0-20。
probability	int	报文发生误码的概率，填写范围 0-100。
crc	int	被修改的报文是否会重新计算crc。1 为是，0 为否。

返回值：

- [HoloWANReturn](#) 对象。

5.3.10 报文乱序（Reordering）

5.3.10.1 普通模式（Normal）

报文乱序：普通模式，设置每个报文被乱序的概率，被乱序的报文会被添加一定的时延，延后发送。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import ReorderingNormal
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的报文乱序参数
path1.l2r.reordering = ReorderingNormal(probability=5, min_delay=11.1, max_delay=99.9)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
proballity	float	每个单报文发生乱序的概率。填写范围 0-20，允许填写至小数点后 3 位 。
min_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最小时间，填写范围 0-10000，允许填写至小数点后 2 位。
max_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最大时间，填写范围 0-10000，允许填写至小数点后 2 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.10.2 抖动模式（Jitter）

报文乱序：抖动模式，通过 6 种曲线模型控制乱序概率的变化。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import ReorderingJitter, ChangeMode
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的报文乱序参数
path1.l2r.reordering = ReorderingJitter(min_delay=11.1, max_delay=99.9,
                                         change_mode=ChangeMode(mode=1, max=99.9,
                                                                    min=11.1, phase=60, period=100))

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
min_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最小时间，填写范围 0-10000，允许填写至小数点后 2 位。
max_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最大时间，填写范围 0-10000，允许填写至小数点后 2 位。
change_mode	ChangeMode	实例对象，报文乱序概率的变化模式。由 6 种曲线模型控制乱序概率的变化。

1.正弦曲线

参数	类型	参数含义
mode	int	mode = 1。
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
phase	int	曲线的初始位置，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

2.峰型曲线

参数	类型	参数含义
mode	int	mode = 2。

参数	类型	参数含义
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
rise	int	曲线的上升部分占曲线的比例，填写范围 0-100。
fall	int	曲线的下降部分占曲线的比例，填写范围 0-100
period	int	曲线变化的周期，填写范围 1-3600。

3.倒钟型曲线

参数	类型	参数含义
mode	int	mode = 3。
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
period	int	曲线变化的周期，填写范围 1-3600。

4.钟型曲线

参数	类型	参数含义
mode	int	mode = 4。
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
period	int	曲线变化的周期，填写范围 1-3600。

5.反Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 5。
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

6.Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 6。

参数	类型	参数含义
max	float	乱序概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	乱序概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

返回值：

- [HoloWANReturn](#) 对象。

5.3.10.3 周期模式（Cycle）

报文乱序：周期模式，每个周期将有若干个报文被乱序。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import ReorderingCycle
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的报文乱序参数
path1.l2r.reordering = ReorderingCycle(type=1, period=999, count=9,
                                       min_delay=11.1, max_delay=99.9)

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
typ	int	报文周期性乱序的模式。1：随机乱序，周期内随机位置的若干个报文将被乱序；2：突发乱序，周期最后的若干个报文将被乱序。
period	int	报文乱序的周期，填写范围 1-9999。
count	int	乱序的报文个数，填写范围 0-9998。
min_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最小时间，填写范围 0-10000，允许填写至小数点后 2 位。

参数	类型	参数含义
max_delay	float	乱序的帧在被重新插入到数据流中相对于原来位置所滞后的最大时间，填写范围 0-10000，允许填写至小数点后 2 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.11 重复帧（Duplication）

5.3.11.1 普通模式（Normal）

重复帧：普通模式，每个报文将有一定的概率被重复。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DuplicationNormal
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的报文重复参数
path1.l2r.duplication = DuplicationNormal(probability=5)
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
probaility	float	每个帧发生重复概率。填写范围 0-10，允许填写至小数点后 3 位。

返回值：

- [HoloWANReturn](#) 对象。

5.3.11.2 抖动模式（Jitter）

重复帧：抖动模式，通过 6 种曲线模型控制报文重复概率的变化。所需参数与 HoloWAN Web GUI 界面一致。

举例：

```
from holowan.v2.engine.path import DuplicationJitter, ChangeMode
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的报文重复参数
path1.l2r.duplication = DuplicationJitter(change_mode=ChangeMode(mode=6, max=99.9,
                                                                    min=9.9, section=50, period=100))

# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
change_mode	ChangeMode	实例对象，重复概率的变化模式。由 6 种曲线模型控制重复概率的变化。

1.正弦曲线

参数	类型	参数含义
mode	int	mode = 1。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
phase	int	曲线的初始位置，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

2.峰型曲线

参数	类型	参数含义
mode	int	mode = 2。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
rise	int	曲线的上升部分占曲线的比例，填写范围 0-100。
fall	int	曲线的下降部分占曲线的比例，填写范围 0-100

参数	类型	参数含义
period	int	曲线变化的周期，填写范围 1-3600。

3.倒钟型曲线

参数	类型	参数含义
mode	int	mode = 3。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
period	int	曲线变化的周期，填写范围 1-3600。

4.钟型曲线

参数	类型	参数含义
mode	int	mode = 4。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
period	int	曲线变化的周期，填写范围 1-3600。

5.反Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 5。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

6.Z型曲线（矩形曲线）

参数	类型	参数含义
mode	int	mode = 6。
max	float	重复概率变化的最大值，填写范围 0.001-20，允许填写至小数点后 3 位。
min	float	重复概率变化的最小值，填写范围 0-19.999，允许填写至小数点后 3 位。
section	int	曲线前半部分占曲线的比例，填写范围 0-100。
period	int	曲线变化的周期，填写范围 1-3600。

返回值：

- [HoloWANReturn](#) 对象。

5.3.12 重定向（Redirect）

重定向功能，将经过该 path 的流量转发到其他 path 上去。

举例：

```
from holowan.v2.engine import Engine

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine = Engine(holowan_ip,holowan_port,engine_id)
path_id = 1

# 获取 path1
path1 = engine.get_path_by_id(path_id)
# 设置 path1 上 port1 到 port2 方向上的流量转发到 path3 上去
path.l2r.redirection = 3
# 应用设置
result = engine.apply_path_configuration(path1)

# 打印 result 判断是否应用成功
print(result)
```

参数：

参数	类型	参数含义
path.l2r.redirection	int	重定向的 path，范围 1-15。请注意不要循环重定向，例如将 path1 的流量重定向到 path3 上，然后又将 path3 上的流量重定向到 path1 上。

返回值：

- [HoloWANReturn](#) 对象。

5.3.13 损伤集合 Impairments

Impairments 类，表示虚拟链路某个方向的损伤集合。该类中包含的损伤类型与 HoloWAN Web GUI 显示内容完全一致。

创建实例对象时，只需指定虚拟链路的方向，链路处于无损状态。

您可以配置 Impairment 类的实例对象，从而应用一组损伤，该损伤可以配置到任何引擎、任何链路的对应方向上。

举例：

```

from holowan.v2.engine.path import Impairments, DelayConstant
from holowan.v2.engine import Engine

impairs = Impairments(direction=1) # port1 -> port2 方向
# 配置损伤
impairs.delay = DelayConstant(delay=100)

# ...
print(impairs)

holowan_ip = "192.168.1.199" # HolowAN 管理端口 IP地址
holowan_port = "8080" # HolowAN 端口号, 默认为 8080
engine1 = Engine(holowan_ip, holowan_port, engine_id=1) # engine 1
engine2 = Engine(holowan_ip, holowan_port, engine_id=2) # engine 2

# 将损伤应用到engine1, path 1 的 port1 -> port2 方向
engine1.apply_impairment(impairs,path_id=1,direction=impairs.direction)

# 将损伤应用到engine2, path 1 的 port2 -> port1 方向
engine2.apply_impairment(impairs,path_id=1,direction=2)

```

参数：

参数	类型	参数含义
direction	int	值为 1 时，表示将损伤应用到虚拟链路的 port1 -> port2 方向。值为 2 时，表示将损伤应用到虚拟链路的 port2 -> port1 方向，值为 3 时，表示将损伤应用到虚拟链路的两个方向。

属性：

- **bandwidth**： 带宽限制。可以是[5.3.1 带宽限制 Bandwidth](#)的任意一种实例对象。默认为：BandwidthFixed(rate=1000,unit=3)。
- **background_utilization**： 背景流量。可以是[5.3.2 背景流量 BackgroundUtilization](#)的任意一种实例对象。默认为：BackgroundUtilizationDisable()。
- **queue_limit**： 队列深度。可以是[5.3.3 队列深度 QueueLimit](#)的任意一种实例对象。默认为：QueueLimitSimple(option=3)。
- **modify**： 报文修改。可以是[5.3.4 报文修改 Modify](#)的任意一种实例对象。默认为：ModifyDisable()。
- **mtu**： 大包分片。可以是[5.3.5 大包分片 MTU](#)的任意一种实例对象。默认为：MTUDisable()。
- **frame_overhead**： 帧开销。可以是[5.3.6 帧开销 FrameOverhead](#)的任意一种实例对象。默认为：FrameOverhead24Ethernet()。
- **delay**： 时延。可以是[5.3.7 时延 Delay](#)的任意一种实例对象。默认为：DelayConstant(delay=0.0)。
- **loss**： 丢包。可以是[5.3.8 丢包 Loss](#)的任意一种实例对象。默认为：LossRandom(loss_rate=0.0)。
- **corruption**： 误码。可以是[5.3.9 误码 Corruption](#)的任意一种实例对象。默认为：BER(error_rate=0, error_rate_index=14, crc=1)。
- **reordering**： 乱序。可以是[5.3.10 报文乱序 Reordering](#)的任意一种实例对象。默认为：ReorderingNormal(probability=0, min_delay=0.1, max_delay=0.5)。

- **duplication**: 重复帧。可以是[5.3.11 重复帧 Duplication](#) 的任意一种实例对象。默认为: DuplicationNormal(probability=0)。
- **redirection**: 重定向。0表示禁用重定向, 1表示开启重定向。默认为: 0 。

方法:

- **Impairments.reset_bandwidth()**
重置带宽限制为默认状态。
- **Impairments.reset_background_utilization()**
重置背景流量为默认状态。
- **Impairments.reset_queue_limit()**
重置队列深度为默认状态。
- **Impairments.reset_modify()**
重置报文修改为默认状态。
- **Impairments.reset_mtu()**
重置大包分片为默认状态。
- **Impairments.reset_frame_overhead()**
重置帧开销为默认状态。
- **Impairments.reset_loss()**
重置丢包为默认状态。
- **Impairments.reset_corruption()**
重置误码为默认状态。
- **Impairments.reset_reordering()**
重置报文乱序为默认状态。
- **Impairments.reset_duplication()**
重置重复帧为默认状态。
- **Impairments.reset()**
重置所有损伤为默认状态。

返回值:

- [HoloWANReturn](#) 对象。

5.3.14 线路选择

5.3.14.1 RouteSelect

线路选择。所需参数与 HoloWAN Web 界面一致。

举例:

```
from holowan.v2.engine import Engine
from holowan.v2.engine.path import (RouteSelect,RouteSelectClient,RouteSelectServer,
                                     RouteSelectNetType,RouteSelectISP
)

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
engine1 = Engine(holowan_ip, holowan_port, engine_id)

# 给 path1 设置线路选择参数
rs = RouteSelect(
    holowan_ip=holowan_ip, holowan_port=holowan_port,
    client=RouteSelectClient.BeiJing, server=RouteSelectServer.ChengDu,
    network_type=RouteSelectNetType.WIFI, isp=RouteSelectISP.CHINA_TELECOM,
    uplink_direction=1
)
# 应用配置
result = engine1.set_route_select(path_id = 1,route_select = rs)

print(result)
```

参数：

参数	类型	参数含义
holowan_ip	str	HoloWAN 管理端口 IP地址。
holowan_port	str	HoloWAN 端口号，默认为 8080。
client	str	客户端。可从预设的 RouteSelectClient 中进行选择。
server	str	服务器。可从预设的 RouteSelectServer 中进行选择。
network_type	str	网络类型。可从预设的 RouteSelectNetType 中进行选择。
isp	str	运营商。可从预设的 RouteSelectISP 中进行选择。
uplink_direction	int	方向。

返回值：

- [HoloWANReturn](#) 对象。

5.3.14.2 RouteSelectClient

线路选择预设的客户端。

属性：

- **BeiJing**： str. 北京市。
- **TianJin**： str. 天津市。

- **HeBei**: str. 河北省。
- **ShanXi**: str. 山西省。
- **NeiMengGu**: str. 内蒙古自治区。
- **LiaoNing**: str. 辽宁省。
- **JiLin**: str. 吉林省。
- **HeiLongJiang**: str. 黑龙江省。
- **ShangHai**: str. 上海市。
- **JiangSu**: str. 江苏省。
- **ZheJiang**: str. 浙江省。
- **AnHui**: str. 安徽省。
- **FuJian**: str. 福建省。
- **JiangXi**: str. 江西省。
- **ShanDong**: str. 山东省。
- **HeNan**: str. 河南省。
- **HuBei**: str. 湖北省。
- **HuNan**: str. 湖南省。
- **GuangDong**: str. 广东省。
- **GuangXi**: str. 广西壮族自治区。
- **HaiNan**: str. 海南省。
- **ChongQing**: str. 重庆市。
- **SiChuan**: str. 四川省。
- **GuiZhou**: str. 贵州省。
- **YunNan**: str. 云南省。
- **XiZang**: str. 西藏自治区。
- **ShaanXi**: str. 陕西省。
- **GanSu**: str. 甘肃省。
- **QingHai**: str. 青海省。
- **NingXia**: str. 宁夏回族自治区。
- **XinJiang**: str. 新疆维吾尔自治区。

5.3.14.3 RouteSelectServer

线路选择预设的服务器。

属性：

- **TianJin**: str. 天津。
- **BeiJing**: str. 北京。
- **ShangHai**: str. 上海。
- **ChongQing**: str. 重庆。
- **ChengDu**: str. 成都。
- **HangZhou**: str. 杭州。
- **NanJing**: str. 南京。
- **FuZhou**: str. 福州。
- **JiNan**: str. 济南。
- **GuangZhou**: str. 广州。

- **HeFei**: str. 合肥。
- **ShenYang**: str. 沈阳。
- **HongKong**: str. 中国香港。
- **XiAn**: str. 西安。

5.3.14.4 RouteSelectISP

线路选择预设的运营商。

属性：

- **CHINA_TELECOM**: str. 中国电信。
- **CHINA_MOBILE**: str. 中国移动。
- **CHINA_UNITED**: str. 中国联通。

5.3.14.4 RouteSelectNetType

线路选择预设的网络类型。

属性：

- **TwoG**: str. 2G。
- **ThreeG**: str. 3G。
- **FourG**: str. 4G。
- **WIFI**: str. WiFi。

6 设置 Admin

6.1 HoloWAN 设置

通过 API 对 HoloWAN 进行管理。如重启设备、获取 HoloWAN 的首选项配置信息等。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号，默认为 8080

# 获取 HoloWAN 的首选项配置
preference = HoloWANAdmin.get_preferences(holowan_ip,holowan_port)

print(preference)
```

返回值：

- [HoloWANReturn](#) 对象。

6.1.1 重启设备

@staticmethod

重启 HoloWAN。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号，默认为 8080

result = HoloWANAdmin.reboot(holowan_ip,holowan_port)

print(result)
```

返回值：

- [HoloWANReturn](#) 对象。

6.1.2 获取设备首选项配置

@staticmethod

获取 HoloWAN 的首选项配置信息。

举例：

```
from holowan.v2 import HolowanAdmin

holowan_ip = "192.168.1.199" # Holowan 管理端口 IP地址
holowan_port = "8080"        # Holowan 端口号，默认为 8080

preference = HolowanAdmin.get_preferences(holowan_ip, holowan_port)
print(preference)
```

返回值：

- Preference 对象。

6.1.3 设置设备首选

@staticmethod

设置 Holowan 的首选项，如修改语言、修改管理网口的默认端口号、修改统计列表的统计频率等。

举例：

```
from holowan.v2 import HolowanAdmin

holowan_ip = "192.168.1.199" # Holowan 管理端口 IP地址
holowan_port = "8080"        # Holowan 端口号，默认为 8080

preference = HolowanAdmin.get_preferences(holowan_ip, holowan_port)
preference.language = 1      # 语言设为中文

# 应用首选项配置
result = HolowanAdmin.set_preferences(holowan_ip, holowan_port, preference)

print(result)
```

返回值： HolowanReturn 对象。

6.1.4 获取 Ethernet 状态信息

@staticmethod

获取 Holowan 的业务网口状态信息，查看业务网口的最大速率与连接状态。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号, 默认为 8080

# 获取 HoloWAN 的网口状态信息
status = HoloWANAdmin.get_ethernet_status(holowan_ip, holowan_port)
print(status)
```

返回值：EthernetStatus 对象。

6.1.5 获取设备的网络配置

@staticmethod

获取 HoloWAN 的网络配置信息，网络配置信息包含管理端口的 IP 地址、MAC 地址、掩码、网关等信息。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号, 默认为 8080

# 获取 HoloWAN 网络配置信息
config = HoloWANAdmin.get_network_configuration(holowan_ip, holowan_port)
print(config)
```

返回值：NetworkConfig 对象。

6.1.6 修改设备的网络配置

@staticmethod

修改 HoloWAN 的网络配置信息。

举例：

```

from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"       # HoloWAN 端口号，默认为 8080

# 修改管理端口的 IPv4 地址
netconfig = HoloWANAdmin.get_network_configuration(holowan_ip,holowan_port)
netconfig.ipv4_address = "192.168.1.111"

# 应用配置
result = HoloWANAdmin.set_network_configuration(holowan_ip,holowan_port,netconfig)

print(result)

```

返回值：HoloWANReturn 对象。

6.2 首选项（Preference）

获取 HoloWAN 的首选项配置。

举例：

```

from holowan.v2 import HoloWANAdmin
from holowan.v2 import Preference

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"       # HoloWAN 端口号，默认为 8080

# 获取 HoloWAN 的首选项配置
preference = HoloWANAdmin.get_preferences(holowan_ip,holowan_port)
print(preference)

```

属性：

- **clean_buffer_when_apply_path_config**: bool类型。下发Path配置时，是否清空buffer，True：是；False：否。该属性可读可写。
- **packets_delay_start_from**: str类型。报文时延起点，取值：receive_time 或 bandwidth_time。该属性可读可写。
- **bandwidth_evaluation**: int类型。带宽计算逻辑，0: 物理模式，1: 软件模式。该属性可读可写。
- **enable_jumbo_frame**: bool类型。巨型帧是否开启，True: 开启，False: 关闭。该属性可读可写。
- **language**: int类型。语言，0: English，1: 中文。该属性可读可写。
- **pixel_capture_size** int类型。pixel 抓包大小。该属性可读可写。
- **printout**: bool类型。是否启用输出。True: 开启，False: 关闭。该属性可读可写。
- **statistics_frequency**: int类型。统计频率。该属性可读可写。
- **http_port**: int/str类型。端口号。可以是整数（如：8080），也可是字符串（如："8080"）。该属性可读可写。
- **gui_version**: int类型。GUI 版本。该属性可读可写。

6.3 Ethernet 状态

获取 HoloWAN 所有业务网口的状态信息。

举例：

```
from holowan.v2 import HoloWANAdmin
from holowan.v2 import EthernetStatus

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080" # HoloWAN 端口号，默认为 8080

# 获取 HoloWAN 所有的业务网口信息
status = HoloWANAdmin.get_ethernet_status(holowan_ip,holowan_port)
print(len(status))
print(status)
```

属性：

- **port_status**: dict. 键为 int 类型的 port_id，值为 port_id 对应网口的状态信息。包含所有网口状态信息，每个网口状态信息为 [PortStatus](#) 实例对象。只读属性。

6.3.1 网口状态 (PortStatus)

HoloWAN 业务网口的状态信息。

举例；

```
from holowan.v2 import HoloWANAdmin
from holowan.v2 import EthernetStatus,PortStatus

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080" # HoloWAN 端口号，默认为 8080

# 获取 HoloWAN 所有业务网口的状态信息
status = HoloWANAdmin.get_ethernet_status(holowan_ip,holowan_port)

# 根据 port ID 获取指定的业务网口的状态信息
port_id = 1
port = status.port_status[port_id]
print(port)
```

属性：

- **port_id**: int类型。网口 ID。port1 网口 ID 为 1，port2 网口 ID 为 2，以此类推。
- **max_speed**: float类型。网口的最大速率。
- **speed_duplex**: str类型。网口连接状态，full-duplex 表示连接正常，linkdown 表示网口未连接。
- **flow_control**: str类型。

6.4 网络配置 (NetworkConfig)

获取 HoloWAN 管理端口的网络配置。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号, 默认为 8080

# 获取 HoloWAN 的网络配置
config = HoloWANAdmin.get_network_configuration(holowan_ip,holowan_port)
print(config)
```

属性：

- **hostname**： str类型。主机名称。
- **mac**： str类型。管理端口的MAC 地址。
- **ipv4_address**： str类型。管理端口的IPv4地址。
- **netmask**： str类型。管理端口的掩码。
- **gateway**： str类型。管理端口的网关地址。
- **ipv4_dhcp_switch**： int类型。IPv4 DHCP 开关。
- **ipv6_global_address**： str类型。IPv6 Global地址。
- **ipv6_link_address**： str类型。IPv6 link地址。
- **ipv6_dhcp_switch**： int类型。IPv6 DHCP 开关。

6.5 时间同步 (Time Sync)

6.5.1 获取 HoloWAN 时间 get_sync_system_time

获取 HoloWAN 当前的系统时间

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号, 默认为 8080

# 获取 HoloWAN 当前的时间
config = HoloWANAdmin.get_sync_system_time(holowan_ip, holowan_port)

print(config)
```

6.5.2 时间同步 set_sync_system_time

将当前控制电脑的时间同步到 HoloWAN 中，需注意，时间同步之后需要重启 HoloWAN，使配置生效。

举例：

```
from holowan.v2 import HoloWANAdmin

holowan_ip = "192.168.1.199" # HoloWAN 管理端口 IP地址
holowan_port = "8080"        # HoloWAN 端口号，默认为 8080

# 同步时间
config = HoloWANAdmin.set_sync_system_time(holowan_ip, holowan_port)

print(config)
```


7. 抓包分析 Pixel

HoloWAN 内置的抓包分析工具 Pixel，可以抓取指定引擎、指定链路上的报文，展示并分析损伤前后的报文。抓取的报文还可导出为 pcap 文件。

7.1 Pixel 过滤器设置

举例：

```
from holowan.v2.pixel import FilterSettings, Pixel, MACFilter, Damage

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 Pixel 过滤规则
fs = FilterSettings(path=1, direction=0)
fs.mac = MACFilter(src="39-A9-FA-D7-F5-E8", dst="any", type="any")
# 设置 Type
fs.type.add_damage(Damage.DAMAGE_AFTER)
fs.type.add_damage(Damage.DAMAGE_LOSS)
# 应用过滤规则
result = pixel.set_filter_settings(fs)

print(result)
```

参数：

参数	类型	参数含义
path	int	待捕获的虚拟链路。
direction	int	待捕获的虚拟链路的方向。

属性：

- **path**： int类型 待捕获的虚拟链路。可读可写。
- **direction**： int类型 待捕获的虚拟链路的方向。可读可写。
- **type**： str 类型 设置抓包时刻和损伤类型。可读可写。字符串的值预设 在 Damage(from holowan.v2.pixel import Damage) 中。
- **mac**： MACFilter 实例对象。可读可写。
- **vlan**： VLANFilter 实例对象。可读可写。
- **ipv4**： IPv4Filter 实例对象。可读可写。
- **ipv6**： IPv6Filter 实例对象。可读可写。
- **tcp_udp_SCTP**： TCPFilter / UDPFilter / SCTPFilter 实例对象。可读可写。

- **packet_length**: LengthFilter 实例对象。可读可写。

7.2 Pixel 设置

HoloWAN 内置的抓包分析工具 Pixel，可以抓取指定引擎、指定链路上的报文，展示并分析损伤前后的报文。

举例：

```
from holowan.v2.pixel import Pixel, FilterSettings

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 Pixel 过滤规则
fs = FilterSettings(path=3, direction=0)
# 应用过滤规则
result = pixel.set_filter_settings(fs)

print(result)
```

参数：

- **holowan_ip**: str. HoloWAN 管理端口 IP地址。
- **holowan_port**: str. HoloWAN 端口号，默认为 8080。
- **engine_id**: int类型 下发损伤配置的引擎ID，engine1 ID 为 1，engine2 ID 为 2，以此类推。

7.2.1 设置过滤器配置 set_filter_settings

设置 Pixel 的过滤器规则。

举例：

```
from holowan.v2.pixel import Pixel, FilterSettings

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 Pixel 过滤规则
fs = FilterSettings(path=3, direction=0)
# 应用过滤规则
result = pixel.set_filter_settings(fs)

print(result)
```

参数：

参数	类型	参数含义
settings	FilterSettings 实例对象	Pixel 的过滤器设置。

7.2.2 开始抓包 start_capture

开始抓包，调用该 API 后 Pixel 开始抓包。当抓包器已满，设备将自动停止抓包。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 开始抓包
result =pixel.start_capture()

print(result)
```

7.2.3 停止抓包 stop_capture

停止抓包，调用该 API 后，Pixel 将停止抓包。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 停止抓包
result =pixel.stop_capture()

print(result)
```

7.2.4 清空抓包器 clean_capture

清空抓包器，调用该 API 后 Pixel 的抓包器将被清空，在清空抓包器之前请注意是否要保存已抓取的包。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 清空抓包器
result = pixel.clean_capture()

print(result)
```

7.2.5 导出 pcap 文件 export_pacp

调用该 API 后，已抓取的报文将导出为 pcap 文件。需要注意导出的报文都是经过过滤器筛选之后的报文。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 pcap 文件的名称以及导出路径
file_path = "/Users/mac/Desktop/test.pcap"
# 导出 pcap文件
result = pixel.export_pacp(file_path=file_path)

print(result)
```

参数：

参数	类型	参数含义
file_path	str	导出 pcap 的文件名以及路径。

7.2.6 设置抓包对象 set_capture_mode

设置抓包对象，抓包对象共 3 类，分别是 ALL_Packets：进入该引擎的所有报文；ALL_Paths：通过所有 Path 的报文；Path1：通过 Path1 的报文。

举例：

```

from holowan.v2.pixel import Pixel, PixelMode

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置抓包对象
result = pixel.set_capture_mode(mode=PixelMode.ONE_PATH, path_id=1)

print(result)

```

参数：

参数	类型	参数含义
mode	str	抓包模式。见 PixelMode
path_id	int	抓包的虚拟链路。当设置 PixelMode.ONE_PATH 模式时，此参数不能为 None。

7.2.7 获取所有 Pixel 文件信息

获取已保存的 Pixel 抓包文件信息，抓取的包可以保存为 pixel 文件。

举例：

```

from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 获取已保存的抓包文件的信息
result = pixel.get_all_pixel_file_info()

print(result)

```

7.2.8 加载 Pixel 文件

将保存的 Pixel 文件加载到 HoloWAN 中。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 加载 pixel 文件
result = pixel.load_pixel_file(file_name="123.pixel")

print(result)
```

7.2.9 释放 Pixel 文件

将已经加载的 pixel 文件释放。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 释放 pixel 文件
result = pixel.release_pixel_file(file_name="123.pixel")

print(result)
```

7.2.10 删除 Pixel 文件

将已保存的 pixel 文件删除。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 删除 pixel 文件
result = pixel.delete_pixel_file(file_name="123.pixel")

print(result)
```

7.3 Pixel 过滤规则

Pixel 的过滤规则。包含的过滤规则和 HoloWAN Web GUI 所呈现的一致。

7.3.1 IPv4 地址过滤规则（IPv4Filter）

通过报文的源 IPv4 地址和目标 IPv4 地址对已抓取的报文进行筛选。

举例：

```
from holowan.v2.pixel import IPv4Filter
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 IPv4 地址过滤规则
fs = IPv4Filter(src="192.168.1.1", smask=32, dst="any", dmask=32, tos="any")
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
src	str	报文的源 IPv4 地址，填写单一的 IPv4 地址或通配符 “any”。
smask	int	源 IPv4 地址掩码，可以填写 1-32。掩码为 24 则表示只匹配 IPv4 地址的前 24 个bit。
dst	str	报文的目标 IPv4 地址，填写单一的 IPv4 地址或通配符 “any”。
dmask	int	目标 IPv4 地址掩码，可以填写 1-32。掩码为 24 则表示只匹配 IPv4 地址的前 24 个bit。
tos	str	TOS 字段，确定报文的优先级。

7.3.2 IPv6 地址过滤规则（IPv6Filter）

通过报文的源 IPv6 地址和目标 IPv6 地址对已抓取的报文进行筛选。

举例：

```
from holowan.v2.pixel import IPv6Filter
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 IPv6 地址过滤规则
fs = IPv6Filter(src="any", dst="any")
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
src	str	报文的源 IPv6 地址，填写单一的 IPv6 地址或通配符 “any”。
dst	str	报文的目标 IPv6 地址，填写单一的 IPv6 地址或通配符 “any”。

7.3.3 报文长度过滤规则（LengthFilter）

通过填写一个长度范围对已抓取的报文进行筛选，报文长度在设定的长度范围内的报文将被选中。

举例：

```
from holowan.v2.pixel import LengthFilter
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置报文长度过滤规则
fs = LengthFilter(length_from=0,length_to=2500)
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
length_from	int	最短报文字节数。

参数	类型	参数含义
length_to	int	最长报文字节数。

7.3.4 MAC 地址过滤规则（MACFilter）

通过报文的源 MAC 地址和目标 MAC 地址对已抓取的报文进行筛选。

举例：

```
from holowan.v2.pixel import MACFilter

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 MAC 地址过滤规则
mac = MACFilter(src="39-A9-FA-D7-F5-E8", dst=r"any", type="any")
# 应用过滤规则
result = pixel.set_filter(mac)

print(result)
```

参数：

参数	类型	参数含义
src	str	报文的源 MAC 地址，填写单一的 MAC 地址或通配符“any”。
dst	str	报文的目标 MAC 地址，填写单一的 MAC 地址或通配符“any”。
type	str	在 MAC 地址规则基础上筛选出指定类型的报文。无需筛选则填写通配符“any”。

7.3.5 TCP 端口号过滤规则（TCPFilter）

通过报文的 TCP 端口号对已抓取的报文进行筛选。

举例：

```
from holwan.v2.pixel import TCPFilter
from holwan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 TCP 端口号过滤规则
fs = TCPFilter(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"], check_version=0)
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
src	int / str / list	报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
dst	int / str / list	报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。

7.3.6 UDP 端口号过滤规则（UDPFilter）

通过报文的 UDP 端口号对已抓取的报文进行筛选。

举例：

```
from holwan.v2.pixel import UDPFilter
from holwan.v2.pixel import Pixel

holwan_ip = "192.168.1.199"
holwan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holwan_ip, holwan_port, engine_id)
# 设置 UDP 端口号过滤规则
fs = UDPFilter(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"], check_version=0)
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
src	int / str / list	报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
dst	int / str / list	报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。

7.3.7 SCTP 端口号过滤规则（SCTPFilter）

通过报文的 SCTP 端口号对已抓取的报文进行筛选。

举例：

```
from holwan.v2.pixel import SCTPFilter
from holwan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 SCTP 端口号过滤规则
fs = SCTPFilter(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"], check_version=0)
# 应用过滤规则
result = pixel.set_filter(fs)

print(result)
```

参数：

参数	类型	参数含义
src	int / str / list	报文的源端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
dst	int / str / list	报文的目标端口号，可以填写：单一的端口号，str 和 int 类型均可；由“端口号-端口号”指定一个端口号范围；通过 list[]设置多个端口号；通配符 “any” 匹配所有端口号。
check_version	int	指定 IPv4 版本或 IPv6 版本，0 表示不选择 IP 协议版本，4 表示 IPv4 协议，6 表示 IPv6 协议。

7.3.8 VLAN 过滤规则（VLANFilter）

通过报文的 VLAN 对已抓取的报文进行筛选。

举例：

```
from holwan.v2.pixel import VLANFilter
from holwan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置 VLAN 过滤规则
vlan = VLANFilter(fpcp="any", fpid="any")
# 设置二层 VLAN 过滤规则
vlan.enable_second_tag(spcp="any", spid="any")
# 应用过滤规则
result = pixel.set_filter(vlan)

print(result)
```

参数：

参数	类型	参数含义
fpcp	str	VLAN 中用于确定优先级的字段，可以填写：0-7 中的任意一个数字；通配符“any”匹配所有的 PCP 字段。
fpid	str	VLAN ID，可以填写：1-4094 中的任意一个数字；通配符“any”匹配所有的 VLAN ID。

方法：

- **VLANFilter.enable_second_tag(spcp:str,spid:str)** 启用二层标签。

7.3.9 Type 损伤类型过滤规则（TypeFilter）

筛选损伤前 或 损伤后的报文，在此基础上，还可叠加损伤类型进行筛选。

举例：

```
from holowan.v2.pixel import TypeFilter, Damage
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

# 创建 TypeFilter
tf = TypeFilter()
# 筛选 损伤前 且报文损伤类型为 bandwidth_drop 的报文
tf.set_capture_moment(Damage.DAMAGE_BEFORE)
tf.add_Damage(Damage.DAMAGE_BANDWIDTH_DROP)

# 应用过滤规则
result = pixel.set_filter(tf)

print(result)
```

参数：

参数	类型	参数含义
capture_moment	int	抓包时刻。0 表示损伤前；1 表示损伤后。默认值为 1。

方法：

- **TypeFilter.set_capture_moment(damage:str)** 设置抓包时刻。可设置 Damage.DAMAGE_BEFORE 或 Damage.DAMAGE_AFTER。
- **TypeFilter.add_damage(damage:str)** 添加损伤类型。
- **TypeFilter.remove_damage(damage:str)** 添加损伤类型。

7.3.10 Direction 方向过滤规则（Pixel.set_filter_direction）

根据报文通过 HoloWAN 的方向进行筛选。如：port1 to port2 方向 以及 port3 to port4 方向均为 left to right。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

# 筛选方向为 从右到左 的报文
result = pixel.set_filter_direction(direction = 1)

print(result)
```

参数：

参数	类型	参数含义
direction	int	0 表示 left -> right；1 表示 right -> left。

7.3.11 Path 过滤规则（set_filter_path）

根据报文通过 HoloWAN 的 Path 进行筛选。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

# 筛选通过 Path1 的报文
result = pixel.set_filter_path(path_id = 1)

print(result)
```

参数：

参数	类型	参数含义
path_id	int	链路 id, Path1 链路 id 为 1，Path2 链路 id 为 2，以此类推。

7.3.12 重置过滤规则（reset_filter_settings）

重置 Pixel 过滤规则，重置后，默认筛选损伤后的报文。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

# 重置过滤规则
result = pixel.reset_filter_settings()

print(result)
```

7.4 设置抓包对象（PixelFormat）

Pixel的抓包模式。共有 3 种抓包模式：所有数据包（ALL_PACKETS）、所有虚拟链路（ALL_PATHS）、单一虚拟链路（ONE_PATH）。

示例：

```
from holowan.v2.pixel import PixelMode, Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)
# 设置抓包对象
res = pixel.set_capture_mode(PixelMode.ALL_PATH)
# 应用过滤规则
result = pixel.set_filter_settings(res)

print(result)
```

属性：

- **ALL_PACKETS**：抓取所有进入该 engine 的数据包。
- **ALL_PATHS**：抓取所有虚拟链路上的数据包。
- **ONE_PATH**：抓取单一的虚拟链路上的数据包。

7.5 获取 pixel 抓包总数（get_pixel_capture_pkt_count）

该接口可获取 Pixel 抓包的包的总个数。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

cnt = pixel.get_pixel_capture_pkt_count()

print(cnt)
```

返回值： int。 -1 表示无数据包或错误。

7.6 获取 pixel 数据包信息（get_pixel_capture_pkt_data）

该接口可将抓到的某个包的数据包信息打印出来。

举例：

```
from holowan.v2.pixel import Pixel

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Pixel
pixel = Pixel(holowan_ip, holowan_port, engine_id)

result = pixel.get_pixel_capture_pkt_data(offset = 0, count = 100)

print(result)
```

参数：

参数	类型	参数含义
offset	int	开始位置。
count	int	数据包数量。

返回值: str.

8. 损伤回放 Playback

损伤回放 Playback 类。Playback 功能可以将由 HoloWAN Recorder 工具录制得到的 txt 回放文件或者用户自定义的 txt 回放文件上传至 HoloWAN，并将 txt 回放文件中设置的损伤应用到指定的 Path。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
```

参数：

参数	类型	参数含义
holowan_ip	str	HoloWAN 管理端口 IP地址。
holowan_port	str	HoloWAN 端口号，默认为 8080。

8.1 上传回放文件

将存储在电脑中的 txt 回放文件上传到 HoloWAN 中。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

# 回放文件在电脑中的路径
uploadFilePath = "C:/Users/Tom/Desktop/test.txt"
# 创建 playback
playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 应用设置
result = playback.upload_playback_file(file_path=uploadFilePath)

print(result)
```

参数：

参数	类型	参数含义
file_path	str	待上传的回放txt文件的路径。

返回值： str。

8.2 获取已上传的回放文件列表

获取 HoloWAN 中的 txt 回放文件列表。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 获取 HoloWAN 中的回放 txt 文件
result = playback.get_playback_file_list()

# 打印 txt 文件列表
print(result)
```

返回值： str。

8.3 删除回放文件

删除 HoloWAN 中的 txt 回放文件。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

# 待删除的 txt 文件名称
filename = "test.txt"
playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 应用设置
result = playback.delete_playback_file(file_name=filename)

print(result)
```

参数：

参数	类型	参数含义
file_name	str	待删除的回放txt文件的名称。

返回值： str。

8.4 获取回放文件内容

获取指定的 txt 回放文件的内容。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1
path_id = 1

# 待获取的 txt 回放文件的名称
filename = "test.txt"
# 仅获取文件前 4096 行数据
is_brief = True
playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 应用设置
result = playback.get_playback_file_data(engine_id=engine_id,path_id=path_id,
                                         file_name=filename,is_brief = is_brief)

print(result)
```

参数：

参数	类型	参数含义
file_name	str	待回放txt文件的名称。
is_brief	bool	是否仅获取前4096行数据（对于比较大的文件，建议开启）。

返回值： str。

8.5 应用回放文件到虚拟链路

将指定的 txt 回放文件应用到指定的虚拟链路。

举例：

```

from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 指定待应用回放文件的 path
path_id = 1
# 指定待应用的回放文件的名称
filename = "test.txt"
playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 应用配置
result = playback.apply_playback_file(engine_id=engine_id,path_id=path_id,file_name=filename)

print(result)

```

参数：

参数	类型	参数含义
engine_id	int	待应用回放txt文件的引擎。
path_id	int	待应用回放txt文件的虚拟链路。
file_name	str	待回放txt文件的名称。

返回值： str。

8.6 释放回放文件

将正在应用的回放文件释放掉，停止回放。

举例：

```

from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 待释放 txt 回放文件的 path
path_id = 1
playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 应用配置，释放 txt 回放文件
result = playback.release_playback_file(engine_id=engine_id,path_id=path_id)

print(result)

```

参数：

参数	类型	参数含义
engine_id	int	待释放回放txt文件的引擎。
path_id	int	待释放回放txt文件的虚拟链路。

返回值： str。

8.7 获取回放状态

获取某个 engine 上某条 path 的回放状态，查看 path 是否在回放。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 待获取回放状态的 engine_id 和 path_id
engine_id = 3
path_id = 1

# 获取当前的回放状态
result = playback.get_playback_status(engine_id=engine_id,path_id=path_id)

print(result)
```

参数：

参数	类型	参数含义
engine_id	int	待获取回放状态的引擎。
path_id	int	待获取回放状态的虚拟链路。

返回值： str。

8.8 设置回放状态

8.8.1 从当前时刻开始或暂停回放

从游标所在的时刻开始或暂停回放，若回放文件是刚刚上传至HoloWAN，则从回放文件的头部开始回放。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 待设置回放状态的 engine_id 和 path_id
engine_id = 3
path_id = 1
action = "play"

# 开始回放
result = playback.set_playback_default_status(engine_id=engine_id,path_id=path_id,
                                              action = action)

print(result)
```

参数：

参数	类型	参数含义
engine_id	int	待设置回放状态的引擎。
path_id	int	待设置回放状态的虚拟链路。
action	int	1 表示 “play”，2 表示 “pause”。
cursor	int	游标位置。

返回值： str。

8.8.2 指定时刻开始或暂停回放

指定游标的时刻，从指定的时刻开始或暂停回放。

举例：


```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 待设置回放状态的 engine_id 和 path_id
engine_id = 3
path_id = 1
action = "play"
cursor = 10

# 在 playback 文件的第 10 条数据的时刻开始回放
result = playback.set_playback_cursor_status(engine_id=engine_id,path_id=path_id,
                                              action=action,cursor=cursor)

print(result)
```

参数：

参数	类型	参数含义
engine_id	int	待设置回放状态的引擎。
path_id	int	待设置回放状态的虚拟链路。
action	int	1 表示 “play”，2 表示 “pause”。
cursor	int	游标位置，指向的是 playback 文件中的第 n 条数据。

返回值： str。

8.9 设置回放开关

设置回放开关，控制 时延、丢包率、带宽是否回放。

举例：

```
from holowan.v2.playback import PlayBack

holowan_ip = "192.168.1.199"
holowan_port = "8080"

playback = PlayBack(holowan_ip=holowan_ip,holowan_port=holowan_port)
# 待设置的 engine_id 和 path_id
engine_id = 3
path_id = 1

# 回放开关列表
switch_list = [0,1,0,1,0,1]
result = playback.set_playback_switch(engine_id=engine_id,path_id=path_id,
                                       switch_list=switch_list)

print(result)
```

参数：

参数	类型	参数含义
engine_id	int	待设置回放状态的引擎。
path_id	int	待设置回放状态的虚拟链路。
switch_list	list	列表长度必须为 6。1 表示开启，0 表示关闭。 各个开关值依次控制bandwidth1,bandwidth2,delay1,delay2,loss1,loss2。

返回值： str。

9. 统计数据

HoloWAN 的数据统计列表数据。可以下载历史数据，清空统计数据，实时获取统计数据等。

9.1 统计数据（Statistics）

HoloWAN 提供了实时的数据统计列表，对报文受到的损伤进行实时统计。

举例：

```
from holowan.v2.engine.stats import Statistics

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 待获取统计列表数据的 path
path_id = 1
# 创建 statistics
stats = Statistics(holowan_ip=holowan_ip, holowan_port=holowan_port, engine_id=engine_id)
# 获取统计列表数据
path_stats = stats.all_paths_statistics_data
# 打印获取到的数据
print(path_stats[path_id].l2r_data)
```

属性：

- **path_statistics_data**: dict. 该引擎下，所有虚拟链路的统计数据。每一条虚拟链路的统计数据为 [PathStatistics](#) 对象。

9.1.1 清理历史统计数据

清理历史统计数据。

举例：

```
from holowan.v2.engine.stats import Statistics

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Statistics
stats = Statistics(holowan_ip=holowan_ip,holowan_port=holowan_port,engine_id)
# 清空统计列表数据
result = stats.clear_data()

print(result)
```

返回值： HoloWANReturn 对象。

9.1.2 将历史数据保存为 csv 文件

将指定虚拟链路的统计数据导出为 csv 文件，开始统计的时间为开机时间或上次清理历史统计数据的时间。

举例：

```
from holowan.v2.engine.stats import Statistics

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Statistics
stats = Statistics(holowan_ip=holowan_ip,holowan_port=holowan_port,engine_id=engine_id)
# 指定的 path
path_id = 1
# 下载的 csv 文件的保存路径
file_path = "data.csv"
# 下载 csv 文件
result = stats.create_csv(path_id=path_id,file_path = file_path)

print(result)
```

返回值： HoloWANReturn 对象。

9.1.3 获取所有 path 的历史数据

获取指定 path_id 的虚拟链路的当前时刻的统计数据。

举例：

```

from holowan.v2.engine.stats import Statistics

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Statistics
stats = Statistics(holowan_ip=holowan_ip, holowan_port=holowan_port, engine_id=engine_id)
# 待获取统计数据的 path
path_id = 1
# 获取统计数据
path_stats = stats.get_path_statistics_data(path_id=path_id)
# 打印获取到的统计数据
print(path_stats)

```

返回值：PathStatistics 对象。

9.2 获取指定虚拟链路的统计数据

获取指定的虚拟链路两个方向的统计数据。

举例：

```

from holowan.v2.engine.stats import Statistics

holowan_ip = "192.168.1.199"
holowan_port = "8080"
engine_id = 1

# 创建 Statistics
stats = Statistics(holowan_ip=holowan_ip, holowan_port=holowan_port, engine_id=engine_id)
# 待获取统计数据的 path
path_id = 1
# 获取统计数据
path1_stats = stats.all_paths_statistics_data[path_id]

# 获取port1 -> port2 方向统计数据
path1_l2r = path1_stats.l2r_data
# 打印获取的数据
print(path1_l2r)

# 根据统计项 key 获取单个统计项
print(path1_l2r["rx_bytes"], path1_l2r["tx_bytes"])

```

属性：

- **l2r_data**：dict. port1 -> port2 方向的统计数据。只读属性。提供了关于传输字节数、接收字节数和其他相关信息的统计数据。
- **r2l_data**：dict. port2 -> port1 方向的统计数据。只读属性。提供了关于传输字节数、接收字节数和其他相关信息的统计数据。

统计项key:

"rx_bytes" 接收字节数，表示接收的总字节数。

"rx_frames" 接收帧数，表示接收的总帧数。

"rx_rate" 接收速率，表示接收数据的实时速率。

"rx_rate_frames" 接收帧速率，表示接收帧的实时速率。

"tx_bytes" 发送字节数，表示发送的总字节数。

"tx_frames" 发送帧数，表示发送的总帧数。

"tx_rate" 发送速率，表示发送数据的实时速率。

"tx_rate_frames" 发送帧速率，表示发送帧的实时速率。

"loss_frames" 丢失帧数，表示在传输过程中丢失的帧数。

"drop_frames" 丢弃帧数，表示由于某些原因（如网络拥塞）而被丢弃的帧数。

"queue_bytes" 队列字节数，表示正在网络队列中等待发送或接收的字节数。

"queue_frames" 队列帧数，表示在网络队列中等待发送或接收的帧数。

"modify_frames" 修改帧数，表示在传输过程中被修改或处理的帧数。

"reordered_frames" 重新排序帧数，表示在传输过程中由于网络延迟或其他因素而重新排序的帧数。

"duplicated_frames" 重复帧数，表示在传输过程中由于某些原因而重复接收或发送的帧数。

"corruption_frames" 误码帧数，表示在传输过程中由于错误或损坏而无法正确接收或发送的帧数。

"background_rx_bytes" 背景流量接收字节数。

"background_rx_frames" 背景流量接收帧数。

"background_tx_bytes" 背景流量发送字节数。

"background_tx_frames" 背景流量发送帧数。

"background_tx_rate" 背景流量发送的实时速率。

"background_drop_frames" 背景流量丢弃帧数。

"cur_bandwidth" 当前带宽，表示当前网络的带宽。

附录一：传输内容解析

注：传输内容格式区分大小写格式

1.1.设备概述信息XML格式

```
<?xml version="1.0" encoding="UTF-8"?>
<ic>
  <eeq>3</eeq>
  <e>
    <ei>1</ei>

    <es>1</es>

    <epq>1</epq>
    <ep>
      <p>
        <pi>1</pi>
        <pn>PATH 1</pn>
        <ie>1</ie>

        <l>1000.0 Mbps - Delay 0.0 ms</l>
        <r>1000.0 Mbps - Delay 0.0 ms</r>
        <re>0</re>

        <rd>0</rd>
      </p>
    </ep>
    <p1 unit='Mbps'>1000</p1>
    <p2 unit='Mbps'>1000</p2>
  </e>
</ic>
```

/* XML 默认Header */
/* Root节点（必须） */
/* 当前设备支持的引擎数量 */
/* 每一个引擎信息概述节点*/
/* Holowan 引擎 ID, engine1 ID 为 1,
engine2 ID 为 2, 以此类推*/
/* 此引擎是否开启：
1. 关闭状态，
2. 开启状态。 */
/* 此引擎下存在的虚拟链路数量 */
/* 引擎所有已存在虚拟链路的概述节点 */
/* 每一个虚拟链路信息概述节点 */
/* 此虚拟链路的ID编号 */
/* 此虚拟链路的名称 */
/* 此虚拟链路是否开启：
1. 关闭状态，
2. 开启状态。 */
/* 此虚拟链路的下行链路损伤信息概述 */
/* 此虚拟链路的上行链路损伤信息概述 */
/* 此虚拟链路的回放功能状态：
0. 关闭状态，
1. 开启状态。 */
/* 此虚拟链路当前的回放下标 */

/* 网口1的最大速率 */
/* 网口2的最大速率 */

1.2.修改虚拟链路配置XML格式

```
<?xml version="1.0" encoding="UTF-8"?>
<holowan_modify_path_config_info>
    <modify_switch></modify_switch>

    <engine_id></engine_id>

    <path_id></path_id>
    <path_name></path_name>
    <if_enbale></if_enbale>

</holowan_modify_path_config_info>
```

/* XML 默认Header */
/* Root节点（必须） */
/* 修改方式：
1. 删除PATH,
2. 添加PATH,
3. 启用 or 禁用PATH。 */
/* HoloWAN 引擎 ID, engine1 ID 为 1,
engine2 ID 为 2, 以此类推 */
/* 指定的虚拟链路的编号，取值范围：1-15编号 */
/* 指定的虚拟链路名称，默认设置PATH_X*/
/* 是否启用该PATH：
1. 禁用PATH, 2. 启用PATH。*/

1.3.修改虚拟链路损伤参数XML格式

```
<?xml version="1.0" encoding="utf-8"?>
<pc>
    <eid></eid>

    <pid></pid>
    <pn></pn>
    <pd></pd>

    <version></version>

    <pltr>
        <bd>
            <s></s>

            <fi>
                <r></r>
                <t></t>
            </fi>
            <ji>
                <r></r>
                <t></t>
            </ji>
            <tb>
                <s></s>

                <air></air>
                <air_t></air_t>

                <bir></bir>
                <bir_t></bir_t>

                <abs></abs>
                <abs_t></abs_t>

                <bbs></bbs>
                <bbs_t></bbs_t>

            </tb>
        </bd>
        <bg>
            <s></s>
```

/* XML 默认Header */
/* Root节点（必须） */
/* HolowAN 引擎 ID, engine1 ID 为 1, engine2 ID 为 2, 以此类推*/
/* 指定的虚拟链路的编号, 取值范围: 1-15编号 */
/* 指定的虚拟链路名称*/
/* 指定的虚拟链路损伤方向:
1. 仅损伤Port1->Port2方向,
2. 仅损伤Port2->Port1方向,
3. 同时损伤双方向。 */
/* XML 版本号, 每条 path 都有独立的版本号, 成功提交配置之后 XML 版本号会自动 +1 */

/* Port1->Port2方向的损伤参数 */
/* 带宽限制配置节点 */
/* 带宽限制类型:
1. 固定模式,
2. 抖动模式,
3. 令牌桶模式。 */
/* 带宽限制固定模式配置节点 */
/* 带宽限制值*/
/* 带宽限制值单位 */

/* 带宽限制抖动模式配置节点 */
/* 带宽限制值 */
/* 带宽限制值单位 */

/* 令牌桶模式配置节点 */
/* 令牌桶算法类型:
1. 单令牌桶算法,
2. 双令牌桶(srTCM)算法,
3. 双令牌桶(trTCM)算法。 */
/* A桶令牌的产生速率 */
/* A桶令牌的产生速率的单位:
1. bps, 2. Kbps, 3. Mbps。 */
/* B桶令牌的产生速率 */
/* B桶令牌的产生速率的单位:
1. bps, 2. Kbps, 3. Mbps。 */
/* A桶可容纳的最大令牌数 */
/* A桶可容纳的最大令牌数的单位:
1. B, 2. KB, 3. MB。 */
/* B桶可容纳的最大令牌数 */
/* B桶可容纳的最大令牌数的单位:
1. B, 2. KB, 3. MB。 */

/* 背景流量配置节点 */
/* 背景流量开关, [1. 关闭, 2. 开启] */

```
<lu></lu>          /* 背景流量带宽抢占比例 */
<bs></bs>          /* 背景流量报文大小 */
</bg>
```

<pre> <ql> <s></s> <dt> <qd></qd> <qdt></qdt> </dt> <red> <w></w> <math></math> <mith></mith> <madp></madp> </red> </ql> <md> <cs></cs> <pt></pt> <po></po> <ps></ps> <pv></pv> <mt></mt> <mo></mo> <ms></ms> <mv></mv> </md> <m> <s>1</s> <n>1500</n> </m> <fo> <t>1</t> <r>24</r> </fo> <d> <s></s> </pre>	<pre> /* 队列限制配置节点 */ /* 队列类型： 1. 简易模式、尾部丢弃算法， 2. RED算法。 */ /* 队列深度值大小 */ /* 队列深度值类型： 1. 报文个数， 2. 内存大小， 3. 时间ms。 */ /* 权重 */ /* 最大阈值 */ /* 最小阈值 */ /* 最大丢包概率 */ /* 修改报文配置节点 */ /* 修改报文开关： 0. 关闭， 1. Normal, 2. Cycle, 3. Random。 */ /* 修改报文二次匹配起始位置类型 */ /* 修改报文二次匹配距离起始位置偏移量 */ /* 修改报文二次匹配值大小： [1, 2, 4, 6, 8], 单位Byte */ /* 修改报文二次匹配值 */ /* 修改报文修改其实位置类型 */ /* 修改报文修改位置距离起始位置的偏移 */ /* 修改报文新的值大小： [1, 2, 4, 6, 8], 单位Byte */ /* 修改报文新的值 */ /* MTU限制配置节点 */ /* MTU限制开关 */ /* MTU限制值 */ /* 以太网帧间隙占用配置节点 */ /* 帧间隙占用类型： 1. 默认以太网24, 2. 最小4, 3. 自定义值。 */ /* 帧间隙大小值 */ /* 时延配置节点 */ /* 时延类型： 1. 常量时延, 2. 平均分布时延, 3. 正态分布时延, 4. Custom时延, </pre>
--	--

<co>	5. Jitter时延抖动,
<de></de>	5. 伽马分布时延,
</co>	7. 累计突发时延。*/
<un>	/* 常量时延配置节点 */
<dmi>0.0</dmi>	/* 时延值, [>0.1], 单位ms */
<dma>50.0</dma>	/* 平均分布时延配置节点 */
<reo>0</reo>	/* 平均分布最小值 */
<shake type_id="0"/>	/* 平均分布最大值 */
</un>	/* 是否允许时延乱序, [0. 关闭, 1. 开启],
<no>	此功能开启则普通乱序无效*/
<de>1.0</de>	/* 平均时延抖动配置节点 */
<me>50.0</me>	/* 正态分布时延配置节点 */
<sd>10.0</sd>	/* 正态分布时延最小截断值 */
<reo>0</reo>	/* 正态分布时延期望值 */
</no>	/* 正态分布时延标准差 */
<cu>	/* 是否允许时延乱序, [0. 关闭, 1. 开启] ,
<made>400</made>	此功能开启则普通乱序无效 */
<mede>200</mede>	/* Custom算法模型 */
<mide>1</mide>	/* 最大时延 */
<pd>40</pd>	/* 平均时延 */
<nd>40</nd>	/* 最小时延 */
<spd>1.58</spd>	/* 最大正向变化值 */
<reo>0</reo>	/* 最大负向变化值 */
</cu>	/* Spread值 */
</d>	/* 是否允许时延乱序, [0. 关闭, 1. 开启] ,
<l>	此功能开启则普通乱序无效 */
<s></s>	/* 丢包配置节点 */
<ra>	/* 丢包类型:
<r></r>	1. 随机丢包,
</ra>	2. 周期丢包,
<cy>	3. 突发丢包,
<a>	4. 双通道丢包,
<l></l>	5. 曲线控制丢包,
</cy>	5. 四状态马尔科夫丢包。 */
<bu>	/* 普通概率丢包配置节点 */
<l></l>	/* 普通概率丢包概率值*/
<mi></mi>	/* 周期丢包配置节点 */
<ma></ma>	/* 周期丢包周期值, 单位为报文个数 */
</bu>	/* 周期丢包数量, 单位为报文个数 */
<du>	/* 突发丢包配置节点 */
<g></g>	/* 突发丢包概率值*/
	/* 突发丢包最小丢包数量 */
	/* 突发丢包最大丢包数量 */
	/* 双通道丢包配置节点 */
	/* 好通道丢包概率 */

	/* 坏通道丢包概率 */
<gtb></gtb>	/* 好通道转换到坏通道的转换概率 */
<btg></btg>	/* 坏通道转换到好通道的转换概率 */
</du>	
<shake type_id="1">	/* 曲线控制丢包配置节点， 属性type_id对应6种不同的曲线模型 */
<max>100</max>	/* 最大丢包率 */
<min>10</min>	/* 最小丢包率 */
<cycle>60</cycle>	/* 抖动的周期 */
<phase>0</phase>	/* 曲线初始位置 */
</shake>	
<mkv>	/* 四状态马尔科夫丢包配置节点 */
<p13></p13>	/* 状态1到状态3的转换概率 */
<p31></p31>	/* 状态3到状态1的转换概率 */
<p32></p32>	/* 状态3到状态2的转换概率 */
<p23></p23>	/* 状态2到状态3的转换概率 */
<p14></p14>	/* 状态1到状态4的转换概率 */
</mkv>	
</l>	
<cor>	/* BER配置节点 */
<s></s>	/* BER类型： 1. 全报文BER， 2. 多范围BER， 3. 包误码。 */
<ber></ber>	/* BER值 */
<beri></beri>	/* BER指数值 */
<range>	/* BER作用范围（仅作用于多范围BER） */
<r1 start="1" end="2"></r1>	/* BER第1个作用范围的起始位置（单位为字节） */
<r2 start="3" end="4"></r2>	/* BER第2个作用范围的起始位置（单位为字节） */
...	
</range>	
</cor>	
<reo>	/* 报文乱序配置节点 */
<s></s>	/* 报文乱序类型： 1. 普通类型， 2. 抖动类型， 3. 周期类型。 */
<no>	
	/* 报文乱序概率 */
<dmi></dmi>	/* 报文乱序延时最小值 */
<dma></dma>	/* 报文乱序延时最大值 */
</no>	
<ji>	
略...	
</ji>	
<cy>	
<type></type>	/* 类型：1.Random类型，2.Burst类型。 */
<period></period>	
<count></count>	
<dmi></dmi>	/* 报文乱序延时最小值 */
<dma></dma>	/* 报文乱序延时最大值 */
</cy>	
</reo>	

```

<du>
    <s></s>

    <p></p>
</du>
</pltr>
<prtl>
    .....

</prtl>
</pc>

```

/* 重复报文配置节点 */

/* 重复报文类型：

1. 默认类型，
2. 抖动类型。 */

/* 重复报文概率 */

/* Port2->Port1方向的损伤参数 */

/* 配置内容与Port1->Port2方向参数一致，
请参考Port1->Port2方向 */

附录：关于抖动的设置（带宽抖动、平均时延抖动，丢包抖动，乱序抖动、重复报文抖动）

```

<shake type_id="">

<max></max>
<min></min>
<cycle></cycle>
<phase></phase>
<proportion></proportion>
</shake>

```

/* 抖动类型：

1. 正弦图像，
2. 斜坡图像，
3. 逆抛物线图像，
4. 抛物线图像，
5. 上升折线图像，
5. 下降折线图像】 */

/* 抖动最大值 */

/* 抖动最小之 */

/* 抖动周期 */

/* 相位偏移值 */

/* 上升或下降曲线，第一部分占比值 */

1.4.修改报文分类器参数XML格式

```
<?xml version="1.0" encoding="utf-8"?> /* XML 默认Header */
<clc> /* Root节点（必须） */
  <engine_id></engine_id> /* HolowAN 引擎 ID, engine1 ID 为 1, engine2 ID 为
                           2, 以此类推 */
  <port> /* 设定Port配置节点, 若多个则包含多个此节点 */
    <port_id></port_id> /* 指定的工作口ID编号 */
    <mac> /* 关于以太网层的报文分类*/
      <src any="1"/> /* 匹配源始MAC地址, 若为任意地址, 则属性any值为1*/
      <dst any="1"/> /* 匹配目标MAC地址, 若为任意地址, 则属性any值为1 */
      <type any="1"/> /* 匹配报文类型, 若为任意类型, 则属性any值为1 */
      <path_id></path_id> /* 命中分类后进入的虚拟链路的编号, 取值范围: 1-15编号 */
    </mac>
    <vlan> /* 关于VLAN层的报文分类*/
      <fpcp any="1"/> /* 匹配第一层VLAN的PCP, 若为任意值, 则属性any值为1*/
      <fpid any="1"/> /* 匹配第一层VLAN的PID, 若为任意值, 则属性any值为1*/
      <enable_stag>1</enable_stag> /* 是否匹配二层VLAN*/
      <spcp any="1"/> /* 匹配第二层VLAN的PCP, 若为任意值, 则属性any值为1 */
      <spid any="1"/> /* 匹配第二层VLAN的PID, 若为任意值, 则属性any值为1 */
      <path_id></path_id> /* 命中分类后进入的虚拟链路的编号, 取值范围: 1-15编号*/
    </vlan>
    <mpls> /* 关于MPLS的报文分类 */
      <label any="1"/> /* 匹配的MPLS Label值, 若为任意值, 则属性any值为1*/
      <path_id></path_id> /* 命中分类后进入的虚拟链路的编号, 取值范围: 1-15编号 */
    </mpls>
    <pppoe> /* 关于PPPoE层的报文分类 */
      <sid any="1"/> /* 匹配的PPPoE的Session ID值, 若为任意值, 则属性any值为1*/
      <code any="1"/> /* 匹配的PPPoE的Code值, 若为任意值, 则属性any值为1 */
      <path_id></path_id> /* 命中分类后进入的虚拟链路的编号, 取值范围: 1-15编号 */
    </pppoe>
    <ipv4> /* 关于网络层IPv4的报文分类 */
      <src any="1"/> /* 匹配的IPv4的源始地址, 若为任意值, 则属性any值为1 */
      <smask>32</smask> /* 源始地址的匹配掩码, 默认32*/
      <dst any="1"/> /* 匹配的IPv4的目标地址, 若为任意值, 则属性any值为1 */
      <dmask>32</dmask> /* 目标地址的匹配掩码, 默认32 */
      <tos any="1"/> /* 匹配的IPv4的TOS值, 若为任意值, 则属性any值为1 */
      <tunnel?flood="1"> /* 添加了一层隧道的识别 */
        <ipv4> /* 使用IPv4规则用于识别隧道内的内容 */
          <src?any="1"/>
          <smask>32</smask>
          <dst?any="1"/>
          <dmask>32</dmask>
          <tos?any="1"/>
        </ipv4>
      </tunnel>
      <path_id></path_id> /* 命中分类后进入的虚拟链路的编号, 取值范围: 1-15编号 */
    </ipv4>
```

<code><ipv6></code>	<code>/* 关于网络层IPv6的报文分类 */</code>
<code> <src any="1"/></code>	<code>/* 匹配的IPv6的源地址，若为任意值，则属性any值为1 */</code>
<code> <dst any="1"/></code>	<code>/* 匹配的IP64的目标地址，若为任意值，则属性any值为1 */</code>
<code> <path_id></path_id></code>	<code>/* 命中分类后进入的虚拟链路的编号，取值范围：1-15编号 */</code>
<code></ipv6></code>	
<code><tcp_udp></code>	<code>/* 关于传输层（TCP or UDP）的报文分类 */</code>
<code> <type>1</type></code>	<code>/* 指定传输层类型，[1. TCP, 2. UDP]*/</code>
<code> <src any="1"/></code>	<code>/* 匹配传输层的源端口号*/</code>
<code> <dst any="1"/></code>	<code>/* 匹配传输层的目标端口号*/</code>
<code> <check>0</check></code>	<code>/* 是否检查传输层类型：</code>
	<code> 0. 不检查，4. 匹配IPv4，5. 匹配IPv6。*/</code>
<code> <path_id></path_id></code>	<code>/* 命中分类后进入的虚拟链路的编号，取值范围：1-15编号 */</code>
<code></tcp_udp></code>	
<code><raw_group></code>	<code>/* 关于报文源始字节匹配的报文分类组*/</code>
<code> <raw></code>	<code>/* 关于报文源始字节匹配的配置节点*/</code>
<code> <type>1</type></code>	<code>/* 匹配字节的大小，[1. 1Byte, 4. 4Byte]*/</code>
<code> <layer>2</layer></code>	<code>/* 匹配起始位置：</code>
	<code> 2. 从以太网层匹配，3. 从网络层匹配。 */</code>
<code> <offset>0</offset></code>	<code>/* 从起始位置向后偏移的字节数*/</code>
<code> <mask>0x0</mask></code>	<code>/* 匹配的掩码*/</code>
<code> <value>0x0</value></code>	<code>/* 匹配的值*/</code>
<code> </raw></code>	
<code> <path_id></path_id></code>	<code>/* 命中分类后进入的虚拟链路的编号，取值范围：1-15编号 */</code>
<code></raw_group></code>	
<code></port></code>	
<code></clc></code>	

1.5.PATH每秒统计数据 XML格式

```
<?xml version="1.0" encoding="utf-8"?>    /* XML 默认Header */
<holowan_result_data_info>                /* Root节点（必须） */
    <path>                                /* 每个PATH的数据节点，若请求多个则存在多个该节点 */
        <time></time>                    /* 记录该点数据的13位时间戳（仅精确到秒） */
        <path_id></path_id>            /* 该数据的PATH ID编号 */
        <left_to_right>                /* left->right方向数据节点 */
            <rx_bytes></rx_bytes>        /* 此时刻累计收到的报文数据大小，单位Byte */
            <rx_frames></rx_frames>      /* 此时刻累计收到的报文数量 */
            <rx_rate></rx_rate>          /* 此时刻收到报文的速率，单位bps */
            <rx_rate_frames></rx_rate_frames> /* 此时刻收到报文的速率，单位frames */
            <tx_bytes>0</tx_bytes>       /* 此时刻累计发送的报文数据大小，单位Byte */
            <tx_frames>0</tx_frames>     /* 此时刻累计发送的报文数量 */
            <tx_rate>0</tx_rate>        /* 此时刻发送报文的速率，单位bps */
            <tx_rate_frames>0</tx_rate_frames> /* 此时刻发送报文的速率，单位frames */
            <loss_frames>0</loss_frames>  /* 此时刻累计损伤导致丢弃报文的数量 */
            <drop_frames>0</drop_frames>  /* 此时刻累计由于带宽限制丢弃的报文数量 */
            <queue_bytes>0</queue_bytes>  /* 此时刻Queue缓存报文数据大小 */
            <queue_frames>0</queue_frames> /* 此时刻Queue缓存的报文数量 */
            <modify_frames>0</modify_frames> /* 此时刻累计修改的报文数量 */
            <reordered_frames>0</reordered_frames> /* 此时刻累计乱序的报文数量 */
            <duplicated_frames>0</duplicated_frames> /* 此时刻累计重复的报文数量 */
            <corruption_frames>0</corruption_frames> /* 此时刻累计BER的报文数量 */
            <background_rx_bytes>0</background_rx_bytes> /* 此时刻累计背景流量进队列的
                                                            报文数据大小，单位Byte */
            <background_rx_frames>0</background_rx_frames> /* 此时刻累计背景流量进队列
                                                            的报文数量 */
            <background_tx_bytes>0</background_tx_bytes> /* 此时刻累计背景流量出队列
                                                            的报文数据大小，单位Byte */
            <background_tx_frames>0</background_tx_frames> /* 此时刻累计背景流量出队列的
                                                            报文数量 */
            <background_tx_rate>0</background_tx_rate> /* 此时刻累计背景流量出队列的
                                                            速率，单位bps */
            <background_drop_frames>0</background_drop_frames> /* 此时刻累计背景流量由于
                                                            带宽限制导致丢包的报文数量 */
        </left_to_right>
        <right_to_left>                /* right->left方向数据节点 */
            .....
        </right_to_left>
    </path>
</holowan_result_data_info>
```

1.5.PATH每秒指定类型统计数据 JSON格式

```
/* id值标记此数据的唯一表示，可用于获取下次数据 */
/* time记录该数据的时间 */
/* rx_rate，该值的含义请查看附录三，数组第一元素表示left->right方向的数据，
   另一表示right->left方向*/
[
  {
    "id": uint64_t,
    "time": uint64_t,
    "rx_rate": [0, 0]
  }
]
```

1.7.设备当前历史数据存在点XML格式

<?xml version="1.0" encoding="utf-8"?>	/* XML 默认Header */
<holowan_history_count>	/* Root节点 */
<engine>	/* 每个引擎节点 */
<engine_id></engine_id>	/* 此HolowAN 引擎 ID, engine1 ID 为 1, engine2 ID 为 2, 以此类推编号 */
<path>	/* 每个虚拟链路节点 */
<path_id></path_id>	/* 此链路的编号，取值范围：1-15编号 */
<current_data_count></current_data_count>	/* 链路当前时刻累计记录数据量*/
</path>	
</engine>	
</holowan_history_count>	

1.8.设备控制口状态XML格式

<?xml version="1.0" encoding="utf-8"?>	/* XML 默认Header */
<holowan_admin_network_config_info>	/* Root节点（必须） */
<network_settings>	/*控制口网络配置节点*/
<hostname>HolowanDev</hostname>	/* Hostname名称*/
<dhcp_switch>off</dhcp_switch>	/* 是否开启DHCP功能，若开启则无需配置IP地址等 */
<ipaddr>192.168.1.233</ipaddr>	/* IP地址 */
<netmask>255.255.255.0</netmask>	/* IP地址掩码 */
<gateway>192.168.1.1</gateway>	/* 网关 */
<dnsserver1>192.168.1.1</dnsserver1>	/* DNS服务器地址1 */
<dnsserver2>114.114.114.114</dnsserver2>	/* DNS服务器地址2 */
</network_settings>	
</holowan_admin_network_config_info>	

1.9.设备业务口状态XML格式

```
<?xml version="1.0" encoding="utf-8"?>          /* XML 默认Header */
<holowan_admin_ethernet_config_info>             /* Root节点（必须） */
  <port>                                           /* 每一个工作口的状态信息节点 */
    <port_id></port_id>                           /* 工作口ID编号 */
    <speed_duplex>linkdown</speed_duplex>         /* 工作口速率信息 */
    <flow_control>linkdown</flow_control>         /* 工作口流控信息 */
  </port>
</holowan_admin_ethernet_config_info>
```

1.10.偏好设置XML格式

```
<?xml version="1.0" encoding="utf-8"?>          /* XML 默认Header */
<holowan_admin_preferences>                       /* Root节点（必须） */
  <clean_buffer></clean_buffer>                   /* 下发Path配置时是否清空buffer,
                                                    取值true或false */
  <zero_line></zero_line>                         /* 报文时延起点,
                                                    取值receive_time或bandwidth_time */
  <enable_jumbo_frame></enable_jumbo_frame >      /* 巨型帧是否开启, 取值0或1
                                                    (0为关闭, 1为开启) */
  <language></language>                           /* Web GUI 界面语言,
                                                    0: English, 1: 中文 */
  <ignore_frame_overhead></ignore_frame_overhead> /* 带宽计算逻辑,
                                                    0: 物理模式（计算帧开销）,
                                                    1: 软件模式（忽略帧开销） */
</holowan_admin_preferences>
```

1.11.获取线路选择信息的JSON格式

```
[
  {
    "client": ["北京市", "天津市", "河北省", ...],
    "server": ["天津", "北京", "上海", ],
    "operator": ["中国电信", "中国移动", "中国联通", ...],
    "NetType": ["2G", "3G", "4G", "WiFi"],
    "code": 0
  }
]
```

1.12.pixel过滤器XML格式

```
<?xml version="1.0" encoding="utf-8"?> /* XML 默认Header */
  <capture_filter> /* Root节点（必须） */
    <mac enable="0"> /* MAC地址过滤，enable为开关，0为关，1为开 */
      <src any="1">any</src> /* 匹配源始MAC地址，若为任意地址，则属性any值为1*/
      <dst any="1">any</dst> /* 匹配目标MAC地址，若为任意地址，则属性any值为1*/
      <type any="1">any</type> /* Ether Type, 详见附录五(5.3), 若为any,
                                则属性any值为1 */
    </mac>
    <ipv4 enable="0"> /* ipv4地址过滤，enable为开关，0为关，1为开 */
      <src any="1">any</src> /* 匹配源始ipv4地址，若为任意地址，则属性any值为1*/
      <dst any="1">any</dst> /* 匹配目标ipv4地址，若为任意地址，则属性any值为1*/
      <smask>32</smask> /* 源始ipv4地址的掩码，取值为32、24、16、8*/
      <dmask>32</dmask> /* 目标ipv4地址的掩码，取值为32、24、16、8*/
      <tos any="1"> /* 通配符，使用通配符any与任意IP地址相匹配，
                                属性any值为1 */
    </ipv4>
    <ipv6 enable="0"> /* ipv6地址过滤，enable为开关，0为关，1为开 */
      <src any="1">any</src> /* 匹配源始ipv6地址，若为任意地址，则属性any值为1*/
      <dst any="1">any</dst> /* 匹配源始ipv6地址，若为任意地址，则属性any值为1*/
    </ipv6>
    <tcp_udp enable="0"> /* tcp/udp报文过滤，enable为开关，0为关，1为开*/
      <type>1</type> /* 协议类型选择， 1为tcp，2为udp*/
      <src any="1">any</src> /* 匹配源tcp端口号，若为any，则属性any值为1 */
      <dst any="1">any</dst> /* 匹配目标tcp端口号，若为any，则属性any值为1 */
      <check>0</check> /* Check Version, 取值：
                                0:Not, 4:ipv4, 6:ipv6。*/
    </tcp_udp>
    <impair_flag enable="1"> /* 损伤前后及损伤类型，enable为开关，0为关，1为关*/
      <after>1</after> /* 损伤后，0为不选择，1为选择 */
      <before>0</before> /* 损伤前，0为不选择，1为选择 */
      <cls_drop>0</cls_drop> /* Classifier Drop, 0为不选择，1为选择 */
      <all>1</all> /* all, 0为不选择，1为选择 */
      <no_impair>0</no_impair> /* No Impairment, 0为不选择，1为选择 */
      <mod>0</mod> /* Modification, 0为不选择，1为选择 */
      <frg_af>0</frg_af> /* 损伤后Fragment, 0为不选择，1为选择 */
      <frg_be>0</frg_be> /* 损伤前Fragment, 0为不选择，1为选择 */
      <dup>0</dup> /* Duplication, 0为不选择，1为选择 */
      <bw_drop>0</bw_drop> /* Bandwidth Drop, 0为不选择，1为选择 */
      <los>0</los> /* Loss, 0为不选择，1为选择 */
      <ber>0</ber> /* Bit-error, 0为不选择，1为选择 */
      <reo>0</reo> /* Reordering, 0为不选择，1为选择 */
      <bypass>0</bypass> /* Bypass, 0为不选择，1为选择 */
    </impair_flag>
    <direction enable="0">0</direction> /* 报文通过的方向，0为Left to Right,
                                1为Right to Left,
                                enable为开关，0为关，1为关 */
```

<code><length enable="0"></code>	<code>/* 报文长度过滤, enable为开关, 0为关, 1为关 */</code>
<code><min>0</min></code>	<code>/* 报文长度范围的最小值 */</code>
<code><max>1500</max></code>	<code>/* 报文长度范围的最大值 */</code>
<code></length></code>	
<code><vlan enable="0"></code>	<code>/* VLAN过滤, enable为开关, 0为关, 1为关 */</code>
<code><fpcp any="1">any</fpcp></code>	<code>/* First Tag PCP, 取值为0到7之间的个位数,</code> <code> 或者为any, 若为any, 则属性any值为1 */</code>
<code><fpid any="1">any</fpid></code>	<code>/* First Tag ID, 取值为0到4095之间的一个单一值,</code> <code> 或者为any, 若为any, 则属性any值为1 */</code>
<code><enable_stag>0</enable_stag></code>	<code>/* Sceond Tag的开关, 0为关, 1为关 */</code>
<code><spcp any="1">any</spcp></code>	<code>/* Second Tag PCP, 取值为0到7之间的个位数,</code> <code> 或者为any, 若为any, 则属性any值为1 */</code>
<code><spid any="1">any</spid></code>	<code>/* Second Tag ID, 取值为0到4095之间的一个单一值,</code> <code> 或者为any, 若为any, 则属性any值为1 */</code>
<code></vlan></code>	
<code><pid enable="0">1</pid></code>	<code>/* 报文通过的链路, 链路取值范围是1-15,</code> <code> enable为开关, 0为关, 1为关 */</code>
<code></capture_filter></code>	

附录二： 错误编号列举

该附录以表格的形式列举了 API 接口调用错误时会出现的各种错误编号。通过错误编号，用户可以快速定位错误发生的位置并做出相应的修改。

2.1 修改虚拟链路配置错误编号

当 [修改性能链路配置](#) 相关接口的参数配置错误时，后端会返回一个负数错误编号。

可以根据以下表格查询到错误码具体的位置以及原因：

错误编号	错误说明
-1	XML没有指定Root节点
-2	modify_switch设定值为空
-3	modify_switch设定值错误
-4	engine_id设定值为空
-5	engine_id设定值错误
-6	path_id设定值为空
-7	path_id设定值错误
-8	path_name设定值为空
-9	path_name设定值错误
-10	if_enbale设定值为空
-11	if_enbale设定值错误
-12	指定删除的PATH不存在
-13	删除PATH失败
-14	最多只能添加15条PATH
-15	指定添加的PATH的已经存在
-16	创建PATH数据节点错误
-17	启用or禁用的PATH不存在

2.2 修改虚拟链路损伤错误编号

当 [修改性能链路（PATH）损伤参数](#) 错误时，后端会返回一串 6 位数字的负数错误码，错误码由 4 部分组成：

- 第 1 位数字：** 功能模块编号，虚拟链路损伤配置功能的编号为 2 。
- 第 2~3 位数字：** 损伤功能模块编号，标识虚拟链路中具体的损伤功能，详见下表2。
- 第 4~5 位数字：** 具体参数编号，标识发生错误的具体参数，详见下表3。
- 第 6 位数字：** 错误原因编号，标识导致错误的具体原因，详见下表1。

可以根据以下表格查询到错误码的具体位置以及对应的错误说明：

表1	错误原因
错误编号	错误说明
1	内容为空
2	内容超出范围
3	当前版本不可用(未授权)
4	类型错误
5	最大值与最小值错误

表2	损伤模块
错误编号	错误说明
00	xml结构错误
01	引擎ID
02	链路ID
03	链路名
04	链路方向
05	bypass功能
06	带宽限制
07	背景流量
08	队列深度
09	MTU
10	帧开销
11	报文修改
12	时延
13	丢包
14	误码率
15	乱序
16	重复报文

表2	损伤模块
17	链路重定向

表3-1	带宽限制参数	
错误编号	具体参数	参数范围
00	bandwidth	-
01	bandwidth property 'enable'	0 or 1
02	bandwidth type	1、2、3
10	bandwidth fixed	-
11	bandwidth fixed rate	1bps~25000Mbps
12	bandwidth fixed rate type	0、1、2
20	bandwidth jitter	-
21	bandwidth jitter rate type	0、1、2
22	bandwidth jitter shake	-
23	bandwidth jitter shake type	0~6
24	bandwidth jitter max	1bps~25000Mbps
25	bandwidth jitter min	less than the maximum
26	bandwidth jitter period	1~3600
27	bandwidth jitter phase	0.00~1.00
28	bandwidth jitter section	0.00~1.00
29	bandwidth jitter rise or fall	0.00~1.00
30	bandwidth token bucket	-
31	bandwidth token bucket type	1、2、3
32	bandwidth token bucket air	0~100000
33	bandwidth token bucket bir	0~100000
34	bandwidth token bucket abs	0~100000
35	bandwidth token bucket bbs	0~100000

表3-2	背景流量	
错误编号	具体参数	参数范围
00	background utilization	-

表3-2	背景流量	
01	background utilization property 'enable'	0 or 1
02	background utilization switch	0 or 1
03	background utilization rate	0.1~100
04	background utilization packet size	60~1500

表3-3	队列深度	
错误编号	具体参数	参数范围
00	queue limit	-
01	queue limit property 'enable'	0 or 1
02	queue limit type	0 or 1
03	queue limit drop tail rate	MAX:1s or 100MB
04	queue limit drop tail unit	0~2
05	queue limit RED weight	0.00 ~ 1.00
06	queue limit RED min threshold	less than the maximum
07	queue limit RED max threshold	0~102400
08	queue limit RED max drop probability	0.00~1.00

表3-4	大包分片	
错误编号	具体参数	参数范围
00	MTU	-
01	MTU property 'enable'	0 or 1
02	MTU switch	1 or 2
03	MTU size	68~9000
04	MTU drop don't fragment packet switch	0 or 1
05	MTU drop oversize packet switch	0 or 1

表3-5	报文修改	
错误编号	具体参数	参数范围
00	modify	-
01	modify property 'enable'	0 or 1
02	modify switch	0~7

表3-5	报文修改	
03	modify match header	1~45
04	modify match offset	0~9000
05	modify match size	1~1024
06	modify match value	hex depend on match size
07	modify header	1~90
08	modify offset	0~9000
09	modify size	1~8
10	modify value	hex depend on match size
11	modify period	1~9999999
12	modify burst	1~9999999
13	modify probability	0.00~100.00
14	modify crc	0 or 1
15	modify checksum	0 or 1
16	modify range	-
17	modify range number	1~10
18	modify range checksum	0 or 1
19	modify range offset	0~9000
20	modify range size	1~1024
21	modify range value	hex string
22	modify range start offset	1~10000
23	modify count	1~10000
24	modify insert	hex string
25	modify exchange	-
26	modify exchange offset	0~9000
27	modify exchange size	0~1024

表 3-6	时延	
错误编号	具体参数	参数范围
00	delay	-
01	delay property 'enable'	0 or 1

表 3-6	时延	
02	delay type	1~7
10	delay constant	-
11	delay constant value	0~60000
20	delay uniform	-
21	delay uniform min	less than the maximum
22	delay uniform max	0~60000
23	delay uniform reordering	0 or 1
24	delay uniform advanced	-
25	delay uniform advanced type	0~6
26	delay uniform advanced max	0~10000
27	delay uniform advanced min	0~10000
28	delay uniform advanced phase	0~1
29	delay uniform advanced period	0~3600
30	delay uniform advanced rise	0.00~1.00
31	delay uniform advanced fall	0.00~1.00
32	delay uniform advanced section	0.00~1.00
40	delay normal	-
41	delay normal min	0~1000
42	delay normal mean	mean + std < 10000
43	delay normal std deviation	mean + std < 10000
45	delay normal reordering	0 or 1
46	delay normal (advanced)	-
47	delay normal (advanced switch)	0 or 1
48	delay normal (advanced period)	0~3600000
49	delay normal (advanced duration)	0~60000
50	delay normal (advanced min)	less than the maximum
51	delay normal (advanced max)	0~10000
60	delay custom	-
61	delay custom (mean)	mean+max_delta < max

表 3-6	时延	
62	delay custom (max)	0 ~ 10000
63	delay custom (min)	less than the mean
64	delay custom (max delta)	mean+max_delta < max
65	delay custom (min delta)	min - min_delta >= 0
66	delay custom (spread)	0.01~100
67	delay custom (reordering)	0 or 1
70	delay jitter	-
71	delay jitter (delay)	0~10000
72	delay jitter (value)	0<jitter+(-)delay<10000
73	delay jitter (type)	1
74	delay jitter (reordering)	0 or 1
80	delay gamma	-
81	delay gamma (shape)	(shape-1)*scale > 10000
82	delay gamma (scale)	(shape-1)*scale > 10000
83	delay gamma (reordering)	0 or 1
90	delay accumulate & burst	-
91	delay accumulate & burst (count)	0~1000
92	delay accumulate & burst (delay)	0~10000
93	delay accumulate & burst (extra delay)	0~10000

表3-7	丢包	
错误编号	具体参数	参数范围
00	loss	-
01	loss property 'enable'	0 or 1
02	loss type	1~7
10	loss random	-
11	loss random rate	0.00~100.00
20	loss jitter	-
21	loss jitter type	1~6
22	loss jitter max	0~100

表3-7	丢包	
23	loss jitter min	less than the maximum
24	loss jitter phase	0.00~1.00
25	loss jitter period	0~3600
26	loss jitter rise	0.00~1.00
27	loss jitter fall	0.00~1.00
28	loss jitter section	0.00~1.00
30	loss cycle	-
31	loss cycle period	1~9999999
32	loss cycle burst	less than the period
40	loss burst	-
41	loss burst probability	0.00~100.00
42	loss burst min	less than the maximum
43	loss burst max	1~9999999
50	loss duo	-
51	loss duo good state probability	0.0~100.0
52	loss duo bad state probability	0.0~100.0
53	loss duo good state change probability	0.0~100.0
54	loss duo bad state change probability	0.0~100.0
60	loss 4state markov	-
61	loss 4state markov p13	0.00~1.00
62	loss 4state markov p31	0.00~1.00
63	loss 4state markov p32	0.00~1.00
64	loss 4state markov p23	0.00~1.00
65	loss 4state markov p14	0.00~1.00
70	loss count	-
71	loss count start offset	0~10000
72	loss count loss count	0~10000

表3-8	误码率	
错误编号	具体参数	参数范围

表3-8	误码率	
00	BER	-
01	BER property 'enable'	0 or 1
02	BER type	1~4
03	BER coefficient	0~9
04	BER index	3~14
05	BER crc	0 or 1
10	BER range	-
11	BER range start offset	less than the end offset
12	BER range end offset	0 ~ 9000
20	BER packet	-
21	BER packet probability	0.0~100.0
22	BER bit number	1~20
30	BER count	1~10000
31	BER count start offset	0~10000

表3-9	报文乱序	
错误编号	具体参数	参数范围
00	reordering	-
01	reordering property 'enable'	0 or 1
02	reordering type	1~3
10	reordering normal	-
11	reordering normal probability	0.0~100.0
12	reordering max	0~10000
13	reordering min	less than the maximum
20	reordering jitter	-
21	reordering jitter type	1~6
22	reordering jitter max	0.0~100.0
23	reordering jitter min	less than the maximum
24	reordering jitter phase	0.0~100.0
25	reordering jitter period	0~3600

表3-9	报文乱序	
26	reordering jitter rise	0.00~1.00
27	reordering jitter fall	0.00~1.00
28	reordering jitter section	0.00~1.00
30	reordering cycle	-
31	reordering cycle type	1 or 2
32	reordering cycle period	0~1000000
33	reordering cycle count	less than the period
34	reordering cycle max	0~10000
35	reordering cycle min	less than the maximum

表3-10	重复报文	
错误编号	具体参数	参数范围
00	duplication	-
01	duplication property 'enable'	0 or 1
02	duplication type	1 or 2
03	duplication probability	0.0~100.0
10	duplication jitter	-
11	duplication jitter type	1~6
12	duplication jitter max	0.0~100.0
13	duplication jitter min	less than the maximum
14	duplication jitter phase	0.00~1.00
15	duplication jitter period	0~3600
16	duplication jitter rise	0.00~1.00
17	duplication jitter fall	0.00~1.00
18	duplication jitter section	0.00~1.00

2.3 修改报文分类器错误编号

当 [修改报文分类器参数](#) 错误时，后端会返回一个负数错误码。通过错误码，用户可以快速定位错误发生的位置并做出相应的修改。

可以根据以下表格查询到错误码具体的位置以及原因：

错误编号	错误说明
-1	XML 错误
-2	ENGINE ID 为空
-3	ENGINE ID 错误
-4	PATH ID 为空
-5	PATH ID 错误
-6	MAC 分类配置错误
-7	VLAN 分类配置错误
-8	PPPoE 分类配置错误
-9	Ipv4 分类配置错误
-10	IPv6 分类配置错误
-11	TCP DUP 分类配置错误
-12	RAW 分类配置错误
-13	组合分类配置错误
-14	分类指定PATH ID 错误

2.4 引擎开关错误编号

错误编号	错误说明
-1	请求方式错误
-2	引擎ID指定错误

2.5 引擎数据清空错误编号

错误编号	错误说明
-1	请求方式错误
-2	引擎ID指定错误

2.6 保存引擎错误编号

错误编号	错误说明
-1	XML 没有 ROOT 节点
-2	engine id 为空

错误编号	错误说明
-3	engine id 错误
-4	engine 开关为空
-5	engine 开关设定错误
-6	engine 现在存在path数量为空
-7	engine 现在存在path数量错误
-8	engine 内 path id 为空
-9	engine 内 path id 错误
-10	engine 内 path 名字为空
-11	engine 内 path 开关为空
-12	engine 内 path 开关错误

2.7 配置控制口错误编号

错误编号	错误说明
-1	请求方式错误
-2	配置XML格式错误

2.8 下发偏好设置错误编号

错误编号	错误说明
-1	不是HTTP POST请求或者XML为空
-2	XML结构解析错误
-3	XML没有root节点
-4	clean_buffer值不是true或false
-5	clean_buffer值为空
-6	zero_line值不是receive_time或bandwidth_time
-7	zero_line值为空
-8	Jumbo Frame开关值不是0或1
-9	Jumbo Frame开关值为空
-10	界面语言选项超出范围
-11	保持设置到设备硬盘失败

附录三：统计数据类型列举

Items	统计项
rx_bytes	接收的数据大小，单位Byte
rx_frames	接收的报文总计数量
rx_rate	接收速率，单位bps
tx_bytes	发送的数据量大小
tx_frames	发送的报文总计数量
tx_rate	发送速率，单位bps
loss_frames	丢弃报文数量
drop_frames	带宽限制导致丢弃报文数量
queue_bytes	Queue队列中缓存的数据大小
queue_frames	Queue队列中缓存的报文数量
modify_frames	被修改的报文数量
reordered_frames	乱序的报文数量
duplicated_frames	重复的报文数量
corruption_frames	BER的报文数量
background_rx_bytes	背景流量总计进队列的数据大小，单位Byte
background_rx_frames	背景流量总计进队列的报文个数
background_tx_bytes	背景流量总计出队列的数据大小，单位Byte
background_tx_frames	背景流量总计出队列的报文个数
background_drop_frames	背景流量由于带宽限制丢弃的报文个数

附录四：损伤参数的配置值范围

Bandwith:

Mode	Rate	Precision
Fixed	bps: 1 ~ 1000000000 Kbps: 0.1 ~ 1000000.0 Mbps: 0.1 ~ 1000.0	Kbps、Mbps支持小数，精度 0.1
Jitter	Max: 0 ~ 1000 Mbps Min: 0 ~ 1000 Mbps < Max Phase: 0 ~ 100 Period: 0 ~ 3600	Max、Min 支持小数，精确度 0.1 Phase、Period不支持小数
Token_Bucket	Type: Single Bucket、Double Bucket (srTCM)、Double Bucket (trTCM) Rate: 0 ~ 1000 Mbps Burst: 0 ~ 1000Mbps CIR: 0.1 ~ 1000Mbps CBS: 1000MB PIR: 0.1 ~ 1000Mbps PBS: 1000MB	CBS、PBS不支持小数 其余参数支持小数，精度 0.1

Background Utilization:

Type	Rate	Precision
Rate	0 ~ 100 %	支持小数，精确度 0.1
Burst	64 ~ 65536	不支持小数

Queue Limit:

Type	Rate	Precision
Drop_Tail	KB: 1 ~ 104857600 Packer: 1 ~ 1638400 ms: 1 ~ 1000	不支持小数
RED	Weight: 0.001 ~ 1 Min Threshold: 0 ~ 102400 Max Threshold: 1 ~ 102400 Max Drop Probability: 0 ~ 100	Max Drop Probability支持小数，精确度 0.001

MTU:

Rate	Precision
68 ~ 9216	不支持小数

Frame Overhead:

Mode	Rate	Precision
Custom	1 ~ 64	不支持小数

Delay:

Mode	Rate	Precision
Constant	0 ~ 10000	支持小数，精确度 0.01
Uniform	min: 0 ~ 10000 < max max: 0.01 ~ 10000 > min	支持小数，精确度 0.01
Normal	min: 0 ~ 10000 < mean mean: 0.01 ~ 10000 > min std: 0.01 ~ 10000 std + mean < 10000 && mean + std > 00.1	支持小数，精确度 0.01
Custom	Delay: 0 ~ 10000 Min.Delay: 0 ~ 10000 < Max.Delay Max.Delay: 0 ~ 10000 > Min.Delay Max.+△: 0 ~ Max.Delay - Delay Max.-△: 0 ~ Delay - Min.Delay Spread: 0.01 ~ 100	支持小数，精确度 0.01
Jitter	Delay: 0 ~ 10000 Jitter: 0 ~ 10000 ≤ Delay Type: Constant、Uniform、Normal	支持小数，精确度 0.01
Gamma	Shape: 0.01 ~ 10000 Scale: 0.01 ~ 10000 Shape * Scale < 10000	支持小数，精确度 0.01
Accumulate & Burst	Count: 0 ~ 1000 Delay: 0.1 ~ 10000 Extra delay: 0 ~ 10000	Delay、Extra delay 支持小数，精确度 0.1

Loss:

Mode	Rate	Precision
Random	0 ~ 100	支持小数，精确度 0.0001
Cycle	0 < Period < 9999999 0 <= Burst < Period	不支持小数
Burst	Probability: 0.001 ~ 100 min: 0 ~ 9999999 < max max: 0 ~ 9999999 > min	Probability支持小数，精确度 0.001
Dule	0.001 <= all <= 100	支持小数，精确度 0.0001
Jitter	Max: 0 ~ 100 Min: 0 ~ 100 < Max Phase: 0 ~ 100 Period: 0 ~ 3600	Max、Min 支持小数，精确度 0.0001 Phase、Period不支持小数
Markov	0 ~ 100	支持小数，精确度 0.0001

BER:

Mode	Limit	Precision
BER	系数: 0 ~ 10 指数: -3 ~ -14	不支持小数
BER-Range	系数: 0 ~ 10 指数: -3 ~ -14 Ranges: 0 ~ 9400	不支持小数
BER-Packet	BER per packet: 1 ~ 20 Probability: 0 ~ 100	Probability 支持小数，精确度 为0.01

Reordering:

Mode	Rate	Precision
Normal	0 <= Probability <= 20 min : 0.1 ~ 10000 < max max : 0.1 ~ 10000 > min	支持小数，精确度 0.001
Jitter	Delay Min: 0.1 ~ 10000 < Delay Max Delay Max: Delay Min ~ 10000 Max: 0.001 ~ 20 Min: 0 ~ 20 < Max Phase: 0 ~ 100 Period: 0 ~ 3600	Delay Min、Delay Max 支持小数，精确度 0.1 Max、Min 支持小数，精确度 0.001

Mode	Rate	Precision
Cycle	Type: Random、Burst 0 < Period < 99999999 0 < Count < Period Delay Min: 0 ~ 10000 < Delay Max Delay Max: Delay Min ~ 10000	Delay Min、Delay Max 支持小数，精确度 0.1

Duplication:

Mode	Rate	Precision
Normal	0 ~ 10	支持小数，精确度 0.001
Jitter	Max: 0.001 ~ 10 Min: 0 ~ 10 < Max Phase: 0 ~ 100 Period: 0 ~ 3600	Max、Min 支持小数，精确度 0.001

附录五： 参数取值列举

5.1 matchType参数取值列举

any	Any
1	Ether
4	Vlan
5	Vlan Stack
6	Mpls
7	PPPoE
8	IPv4
12	IPv6
15	TCP
17	UDP

5.2 modifyType参数取值列举

any	Any
1	Packet
1	Ether Head
4	Vlan Head
5	Vlan Stack Head
6	Mpls Head
7	PPPoE Head
8	IPv4 Head
12	IPv6 Head
15	TCP Head
17	UDP Head
46	Ether Payload
49	Vlan Payload
50	Vlan Stack Payload

any	Any
51	Mpls Payload
52	PPPoE Payload
53	IPv4 Payload
57	IPv6 Payload
60	TCP Payload
62	UDP Payload

5.3 EtherType参数取值列举

any	Any
0x0800	IPv4
0x0806	ARP
0x0842	Wake-on-LAN
0x22F0	AudioVideoTransport
0x22F3	IETF TRILL
0x6003	DECnet Phase IV
0x8035	RARP
0x809B	AppleTalk
0x80F3	AARP
0x8100	VLAN-tagged frame
0x8137	IPX37
0x8138	IPX38
0x8204	QNX Qnet
0x86DD	IPv6
0x8808	Ethernet flow control
0x8809	Slow Protocols
0x8819	CobraNet
0x8847	MPLS unicast
0x8848	MPLS multicast
0x8863	PPPoE Discovery Stage

any	Any
0x8864	PPPoE Session Stage
0x8870	Jumbo Frames
0x887B	HomePlug
0x888E	EAP over LAN
0x8892	PROFINET Protocol
0x889A	HyperSCSI
0x88A2	ATA over Ethernet
0x88A4	EtherCAT Protocol
0x88A8	Provider Bridging
0x88AB	Ethernet Powerlink
0x88CC	Link Layer Discovery
0x88CD	SERCOS III
0x88E1	HomePlug AV MME
0x88E3	Media Redundancy Protocol
0x88E5	MAC security
0x88F7	Precision Time Protocol
0x8902	CFM
0x8906	FCoE
0x8914	FCoE Initialization
0x8915	RoCE
0x892F	HSR
0x9000	Ethernet Configuration Testing
0xCAFE	LLT

附录六：HoloWAN Python API 脚本示例

example 1 设置损伤

```
# 导入相关 module
from holowan.v2.engine import Engine
from holowan.v2.engine import Sequential
from holowan.v2.engine.path import (Path, BandwidthTokenBucket, QueueLimitRED, Impairments,
                                     BackgroundUtilizationRandom, ModifyRandom, DelayNormal,
                                     LossBurst, BERRange, ChangeMode, DuplicationJitter)

# holowan 相关信息
holowan_ip = "192.168.1.111"
holowan_port = "8080"

engine_id = 3
path_id = 1

# 创建 Engine 实例对象时，将从后端获取对应 engine 的信息与配置（状态1）
engine1 = Engine(holowan_ip, holowan_port, engine_id)
# 如果在创建了 engine1 后，在 Web GUI 上手动对引擎进行了配置（状态2）
# 那么 engine1 将不是最新的“状态2”，仍是“状态1”
# 如果需要获取引擎最新的状态，请使用 Engine.update()方法
engine1.update()

# 重置引擎
result = engine1.reset_engine()
if not result:
    print(result.err_code, result.err_msg, result.err_reason)

# 保存引擎
result = engine1.save_engine()
print(result)

# 设置单个损伤
token_bucket = BandwidthTokenBucket(type=3, cir=10, cir_unit=3, cbs=10, cbs_unit=3,
                                     pir=10, pir_unit=3, pbs=50, pbs_unit=3)
result = engine1.apply_impairment(token_bucket, path_id=1, direction=1)
print(result)

# 设置一组损伤(同时设置多个损伤)
impair1 = Impairments(1)
# 带宽
impair1.bandwidth = BandwidthTokenBucket(type=3, cir=10, cir_unit=3, cbs=10, cbs_unit=3,
                                           pir=10, pir_unit=3, pbs=50, pbs_unit=3)

# 背景流量
impair1.background_utilization = BackgroundUtilizationRandom(rate=5, burst=66)
# 队列深度
impair1.queue_limit = QueueLimitRED(weight=0.003, min_threshold=8, max_threshold=22,
```

```

probability=0.03)

# 报文修改
impair1.modify = ModifyRandom(match_header=1,match_offset=1,match_size=1,match_value="0x0",
                              modify_header=1,modify_offset=0,modify_size=1,
                              modify_value="0x1",random_rate=0.1,crc=1,checksum=1)

# 时延
dnormal = DelayNormal(minimum=1.1,mean=55.5,std_deviation=9.9,enable_reordering=1)
dnormal.enable_advanced_setup(period=61,duration=2,min=899,max=999)
impair1.delay = dnormal

# 丢包
impair1.loss = LossBurst(probability=5,min=10,max=19)

# 误码
impair1.corruption = BERRange(bit_error_rate=1,bit_error_rate_index=13,crc=1,
                              range_list=["1-10","11-20"])

# 重复帧
impair1.duplication = DuplicationJitter(change_mode=ChangeMode(max=10,min=1,rise=0.29,
                                                                fall=0.19,period=59,mode=2))

# 将上述损伤参数应用到 path1 的 port1 to port2 方向
result = engine1.apply_impairment(impair1,path_id=1,direction=1)
print(result)

# 修改某个损伤的参数
# 方式一（不建议）
path1 = engine1.get_path_by_id(path_id=1)
# 这种方式只有当您确定该损伤的具体类型时，才可用
# 否则，此方式很有可能造成错误的结果
path1.l2r.delay.minimum = 1.123
result = engine1.apply_path_configuration(path1)
print(result)

# 方式二（通过损伤的实例对象和设置单个损伤的方法）
dnormal.minimum = 1.123
result = engine1.apply_impairment(dnormal,path_id=1,direction=1)
print(result)

```

example 2 设置报文过滤规则

```
from holowan.v2.engine import Engine
from holowan.v2.engine import Sequential
from holowan.v2.engine.classifier import RawByteRule, CombinationRule, TCPRule, \
    SCTPRule, MACRule, PPPoERule, MPLSRule, VLANRule
```

```
# HolowAN 信息
```

```
holowan_ip = "192.168.1.111"
```

```
holowan_port = "8080"
```

```
engine_id = 3
```

```
# 获取引擎
```

```
engine1 = Engine(holowan_ip, holowan_port, engine_id)
```

```
...
```

因为 HolowAN 默认状态 (reset engine状态) 时, 默认会有一条 MACRule,
为了避免这条默认规则的影响, 先清空报文分类器, 再增加分类规则

```
...
```

```
# 清空报文分类器
```

```
engine1.packet_classifier.reset()
```

```
#####
```

```
# 每次添加单条分类规则
```

```
# 增加一条 RAW 1 Byte 报文分类规则
```

```
raw1 = RawByteRule(type=1, action=1)
```

```
raw1.add_raw_byte(layer=2, offset=0, mask="0x0", value="0x0")
```

```
raw1.add_raw_byte(layer=2, offset=0, mask="0x0", value="0x0")
```

```
# 增加一条组合分类规则
```

```
comb = CombinationRule(action=1)
```

```
# 组合分类规则的子规则将按照参数传入顺序
```

```
comb.add_rule(
```

```
    TCPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
            check_version=0, action=1),
```

```
    SCTPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
            check_version=0, action=1),
```

```
    MACRule(src="any", dst="any", type="any", action=1),
```

```
    PPPoERule(sid="any", code="any", action=1),
```

```
    MPLSRule(label="any", action=1)
```

```
)
```

```
# 应用报文分类器的改动
```

```
result = engine1.apply_classifier_changes()
```

```
print(result)
```

```
#####
```

```
# 同时设置多条分类规则
```

```
# 使用Sequential容器, 规则将按照参数传入顺序放入容器中
```

```
# 然后将其赋给报文分类器的 port, 其中port1表示HolowAN 的port 1,3,5,...;
```

```
# port2表示HolowAN 的port 2,4,6,...
```

```
engine1.packet_classifier.port1 = Sequential(
```

```

    TCPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
            check_version=0, action=1),
    SCTPRule(src=[123, "456", "578-999"], dst=["12", 56, "589-999", "any"],
            check_version=0, action=1),
    MACRule(src="any", dst="any", type="any", action=1),
    PPPoERule(sid="any", code="any", action=1)
)

# PacketClassifier.add_rule 方法添加到容器末尾
vlan = VLANRule(fpcp="any", fpid="any", action=1)
engine1.packet_classifier.add_rule(1, vlan)
# 应用报文分类器的改动，并打印结果
print(engine1.apply_classifier_changes())

#####

# 设置一条分类规则（该规则将以追加的方式设置，即添加到末尾）
# 添加 raw byte 1 规则到 port 2
result = engine1.apply_rule_to_classifier(raw1,port=2)
print(result)

#####

# 在指定位置插入一条分类规则
engine1.packet_classifier.insert_rule(port=1,index=0,rule=TCPRule(src=[123, "456", "578-999"],
                                                                    dst=["12", 56, "589-999", "any"], check_version=0,
                                                                    action=1))
print(engine1.apply_classifier_changes())

#####

# 修改一条分类规则
engine1.packet_classifier.modify_rule_by_idx(1, 0, VLANRule(fpcp="any", fpid="any", action=1))
result = engine1.apply_classifier_changes()
print(result)

#####

# 将分类规则重新排列（设置报文优先级）
engine1.packet_classifier.rearrange_rules(port=1, order=[0, 3, 1, 2])
result = engine1.apply_classifier_changes()
print(result)

#####

# 删除指定索引位置的规则
engine1.packet_classifier.remove_rule(port = 2,index = 0)
print(engine1.apply_classifier_changes())

```